

Sprint Report

Portfolio Task 2/3/4

Unit code: COS40005/EAT40005

Unit name: Computing/Engineering Technology Project A

Submission date: 28th of November, 2025

Group 7:

Daniel Tiong (102777801)

Alif Harriz (102782711)

Ashley Jong (102780087)

Basil Agas (102778888)

Edison Ho (102779496)

1. Contribution Summary (This Sprint)

All team members should complete the following table together. You can provide a yes/no response or a brief comment where appropriate.

Team Member	Daniel Tiong	Alif Harriz	Ashley Jong	Basil Agas	Edison Ho
Contribution Finished tasks in time or on time with an acceptable quality	Yes	Yes	Yes	Yes	Yes
Initiatives Self-assigned tasks and proactively found solutions to problems	Yes	Yes	Yes	Yes	Yes
Communication Attended all team meetings	Yes	Yes	Yes	Yes	Yes
Communication Attended all supervisor meetings	Yes	Yes	Yes	Yes	Yes
Communication Attended all client meetings	Yes	Yes	Yes	Yes	Yes
Communication Replied every time within an acceptable timeframe	Yes	Yes	Yes	Yes	Yes
Communication	Yes	Yes	Yes	Yes	Yes

Kept the team updated about the status					
Respect The student respected others and listened to different opinions	Yes	Yes	Yes	Yes	Yes

2. Contribution Details (This Sprint)

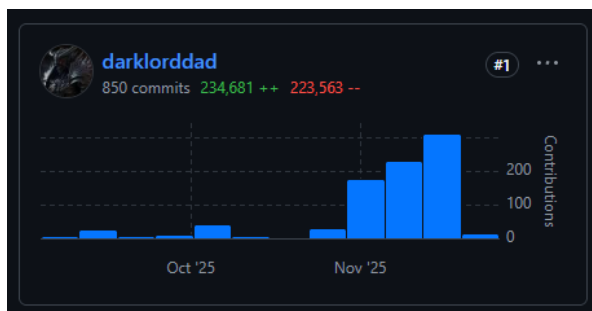
In this section, every member of the team should write a summary of what they have contributed during this sprint. All contributions must be supported by evidence, such as the number of GitHub commits, a screenshot of developed screens, a summary/outcome of research, etc.

2.1. Daniel (Project Lead and AI Engineer)

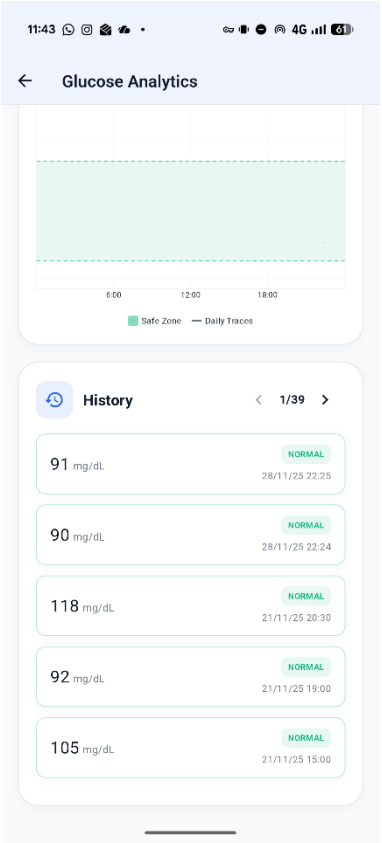
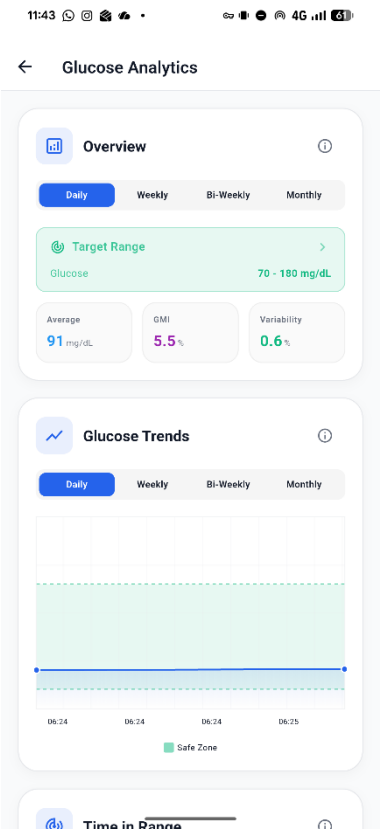
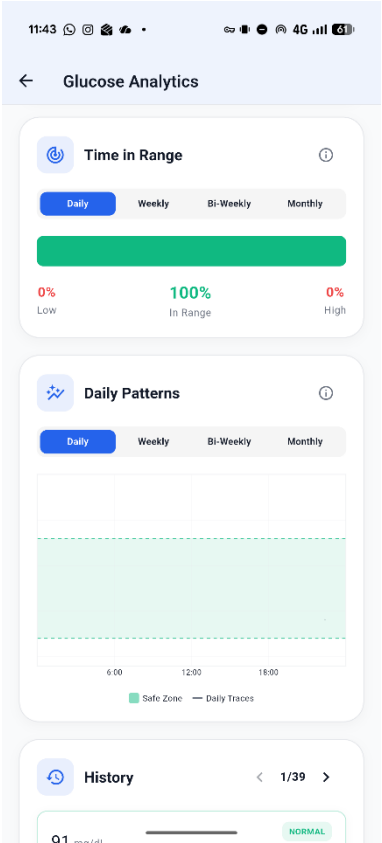
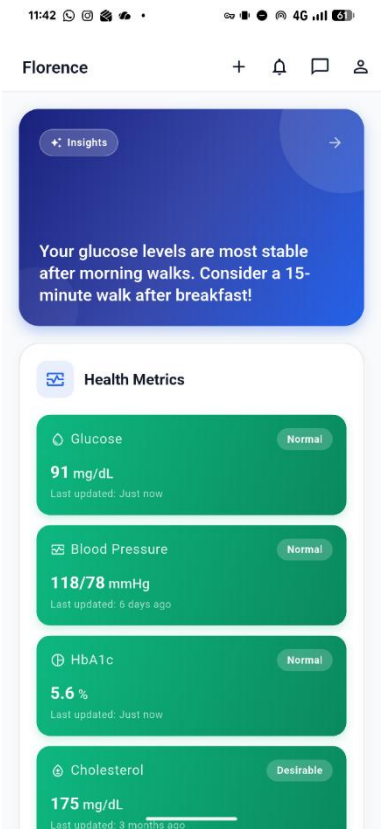
Planned: Work on LangChain Agents.

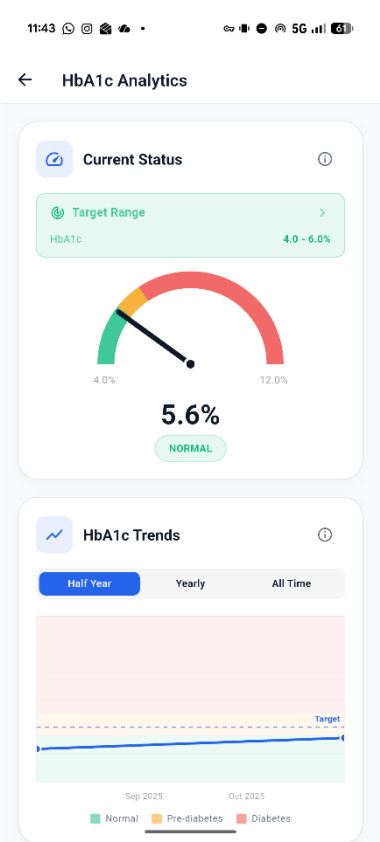
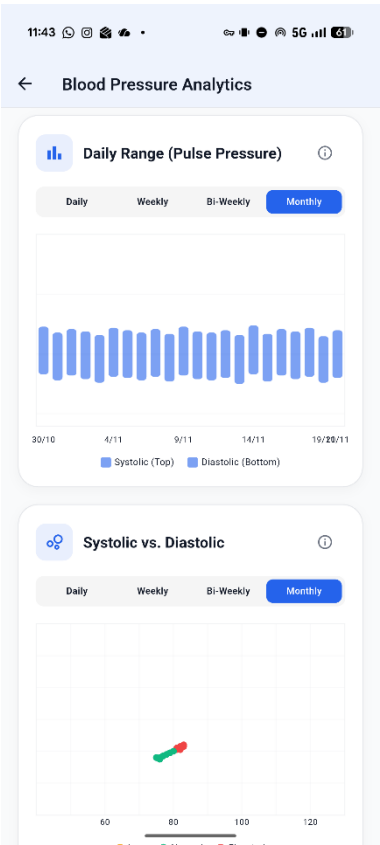
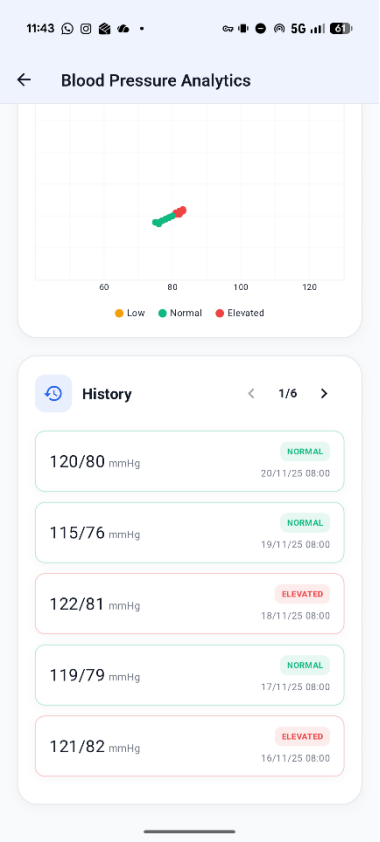
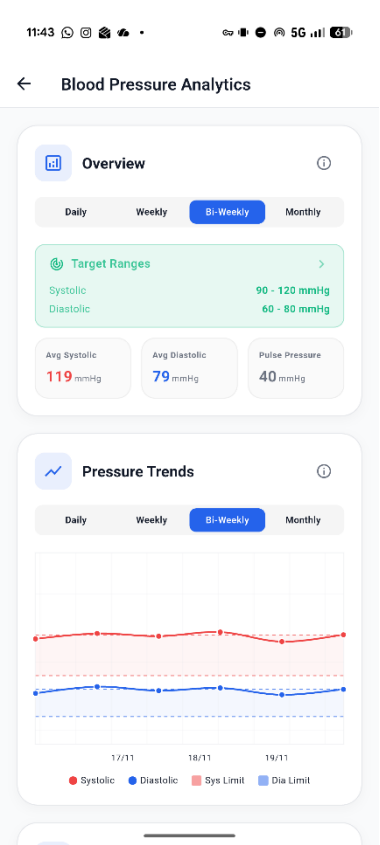
Actual: I pivoted to **Integration and Architecture**. I deleted legacy AI files and focused on making the patient dashboard fully functional with our backend service. This aligns perfectly with the “Strategic Pivot” narrative in the report.

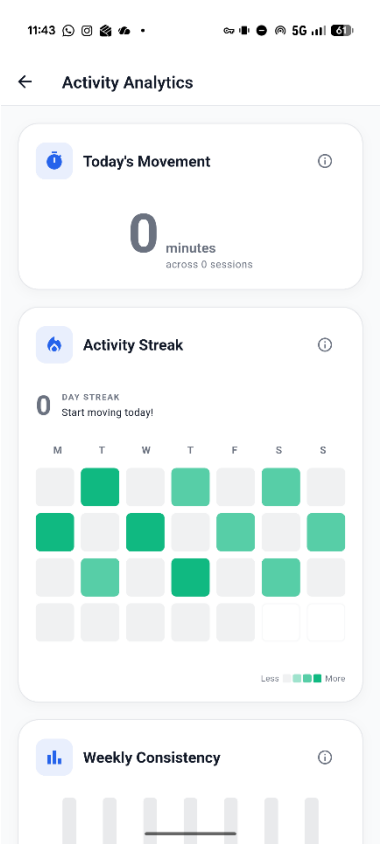
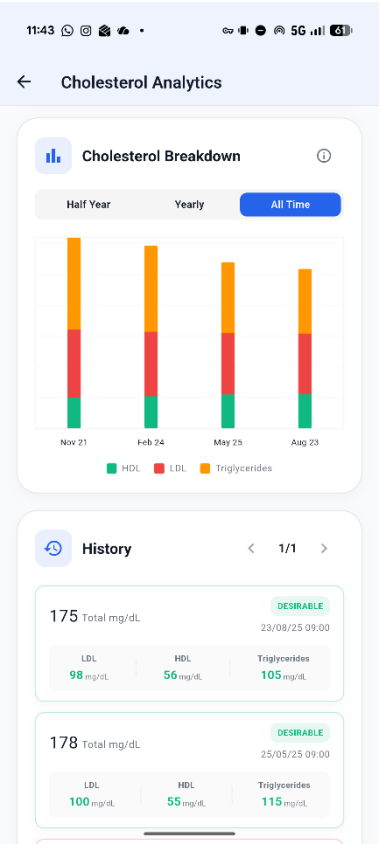
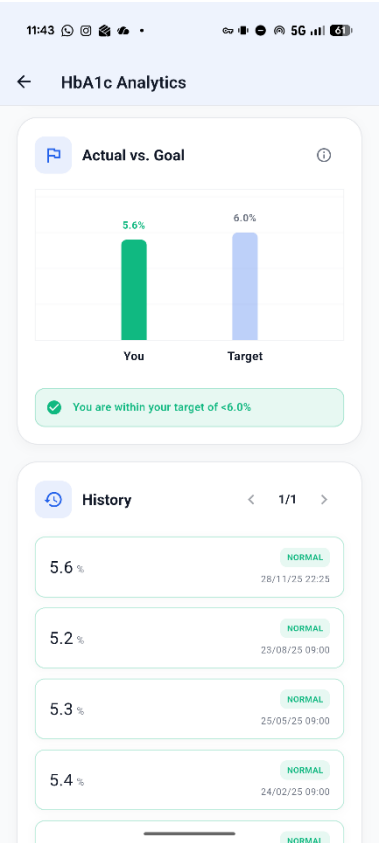
Evidence of Work

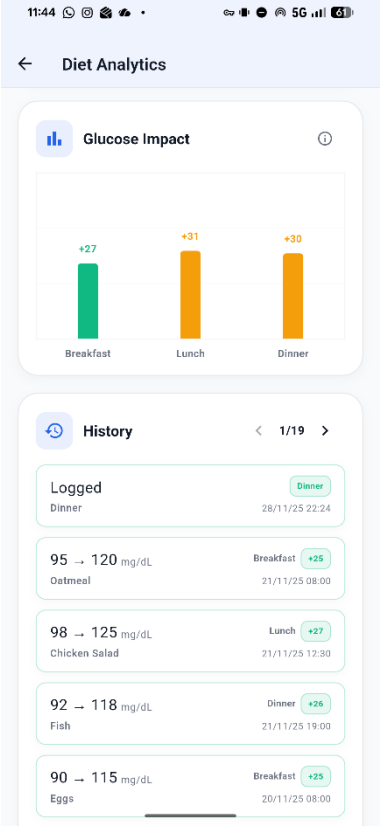
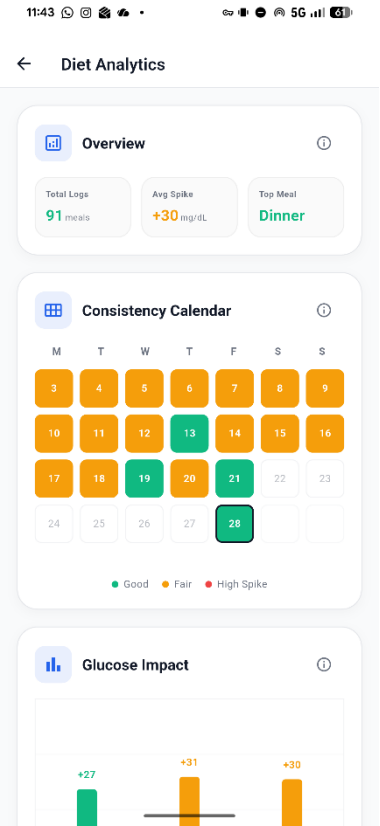
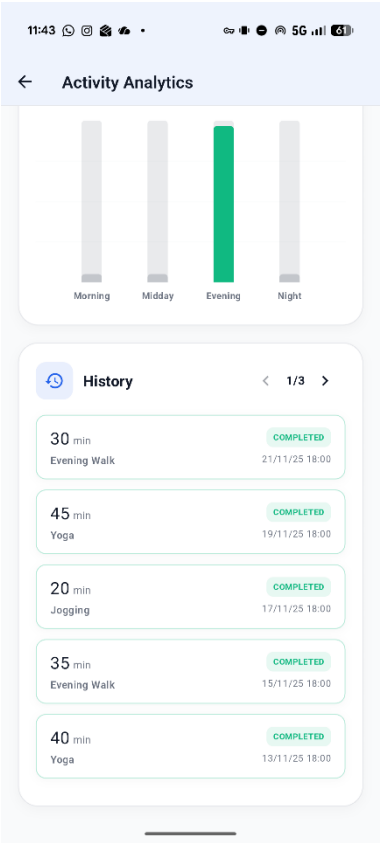
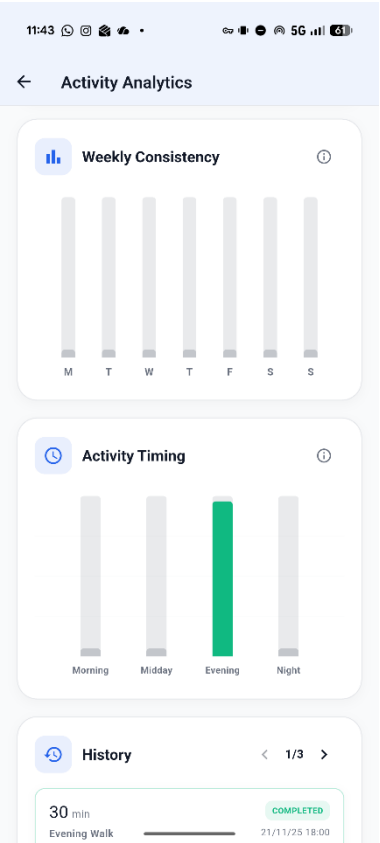


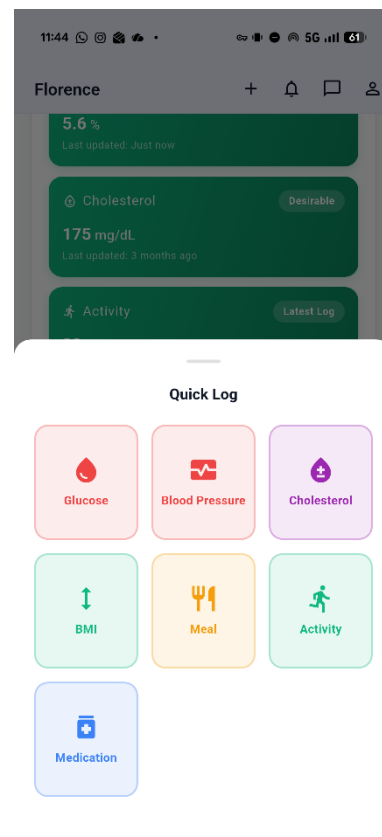
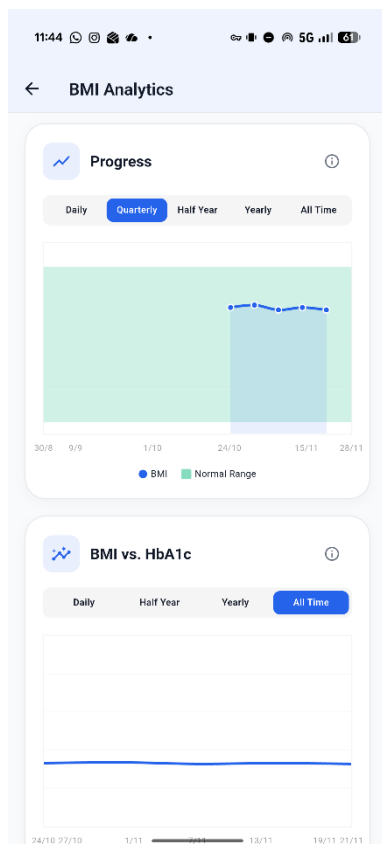
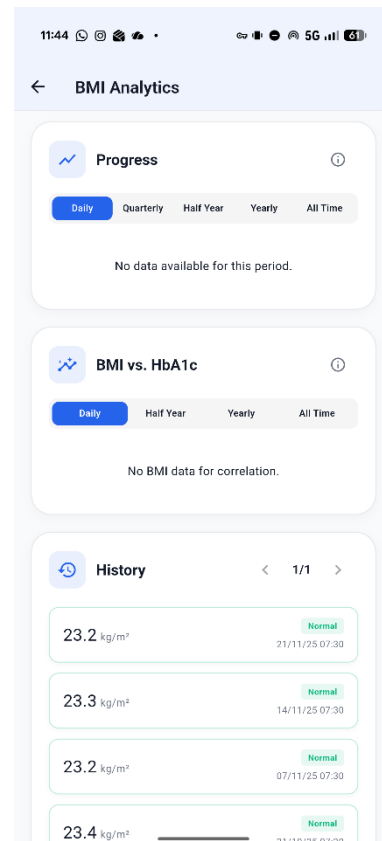
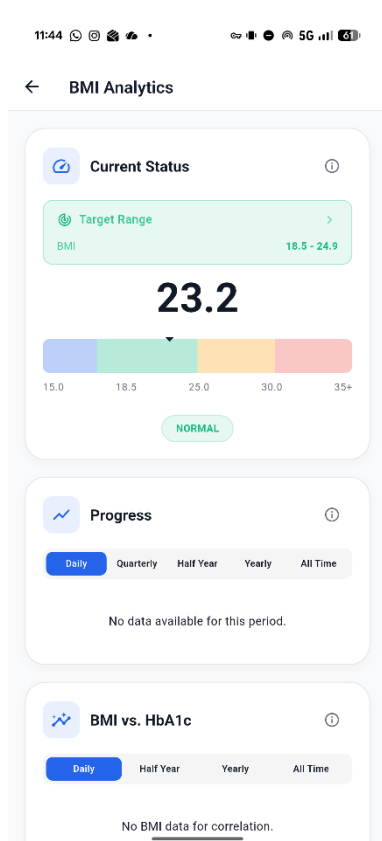
Screenshot of the Application

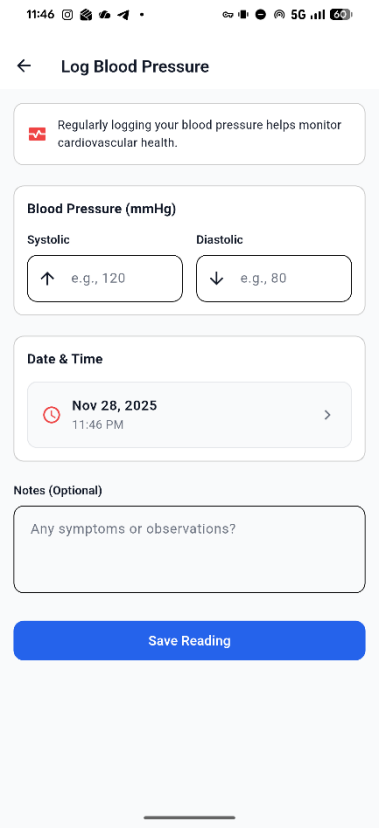
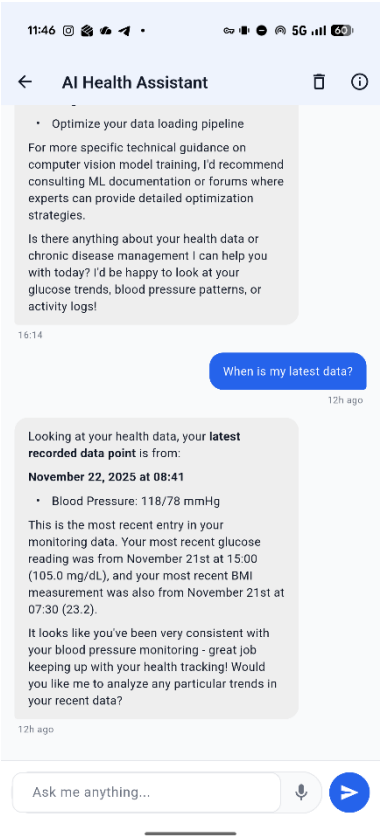
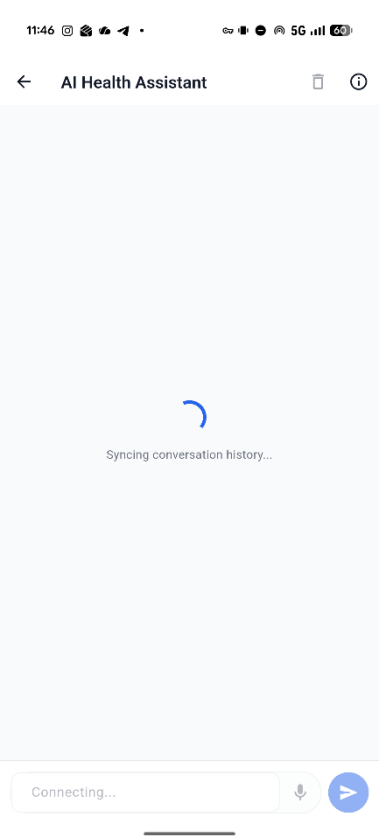












2.2. Alif (Frontend Developer - Patient Dashboard)

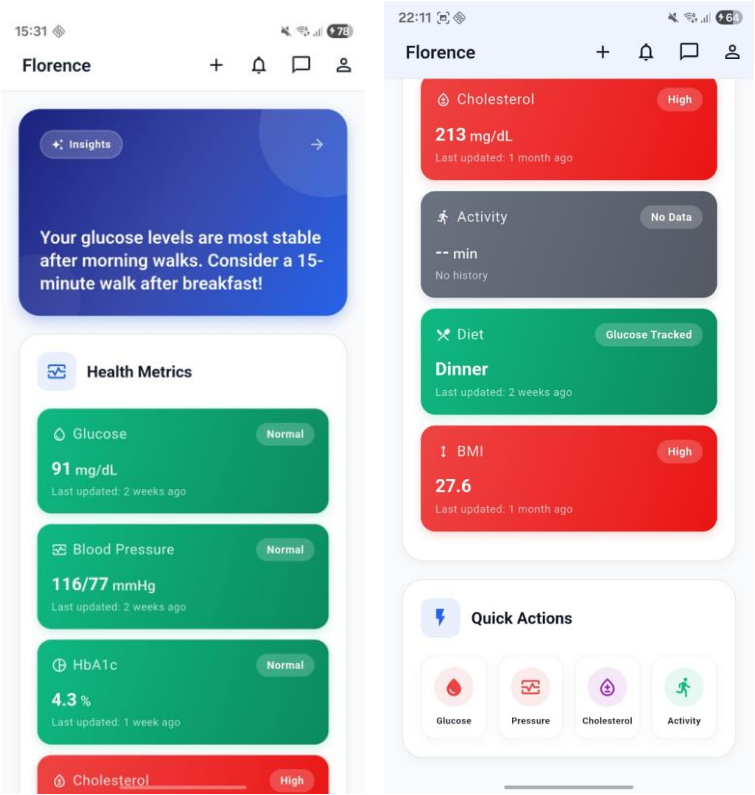
Evidence of Work

GitHub Commits

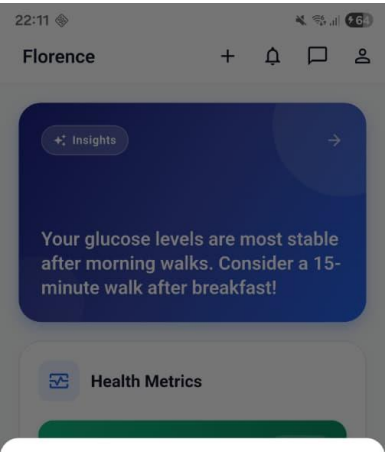
main		Harrisau	All time
Commits on Nov 18, 2025			
Update routes and minor adjustment to tech_stack.svg		a0d8e55	<>
Harrisau committed last week · 2 / 5			
System Architecture svg font change		a1e3683	<>
Harrisau committed last week			
Merge pull request #40 from darklorddad/claude/design-showcase-poster-01R382zh5xcJ9FQn7KFTGei		740ac91	<>
Harrisau authored last week · 4 / 5		Verified	
Merge pull request #39 from darklorddad/claude/design-showcase-poster-01R382zh5xcJ9FQn7KFTGei		d355516	<>
Harrisau authored last week · 4 / 5		Verified	
Minor adjustments to login_screen		95c99de	<>
Harrisau committed last week · 4 / 5			
Delete patient dashboard ai		898a4ac	<>
Harrisau committed last week			
Update routing navigation		e0c9df2	<>
Harrisau committed last week · 4 / 5			
Merge branch 'main' of https://github.com/darklorddad/Swinburne-COS40005-CTPA-FLORENCE		d1d4db9	<>
Harrisau committed last week · 4 / 5			
Integrate pubspec.yaml		396d01c	<>
Harrisau committed last week			
Move clinician_dashboard to features folder		46918a1	<>
Harrisau committed last week			
Add google_font for dependency		d1b844f	<>
Harrisau committed last week			
chore: add project structure txt file for AI context		804f13d	<>
Harrisau committed last week			
Commits on Nov 15, 2025			
fix: remove unused assets declaration from pubspec.yaml		ec07cb3	<>
Harrisau committed 2 weeks ago · 1 / 2			
Commits on Nov 11, 2025			
Merge changes from FLORENCE_Digital_Health_Platform-Implementation-AI_patient-s-side		fb19918	<>
Harrisau committed 2 weeks ago · 2 / 2			
Delete florence/platform_service directory		0aca131	<>
Harrisau authored 2 weeks ago · 2 / 2		Verified	

Screenshot of Application

Patient Homepage



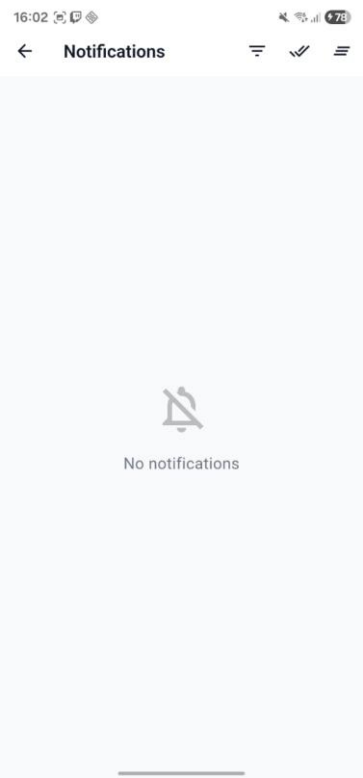
Patient Quick Log



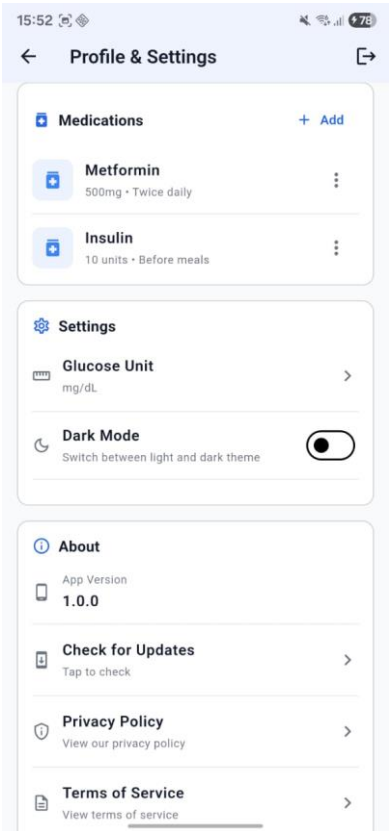
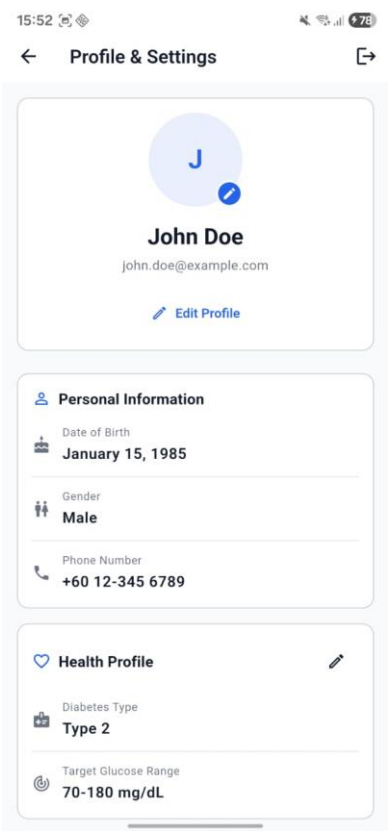
Quick Log



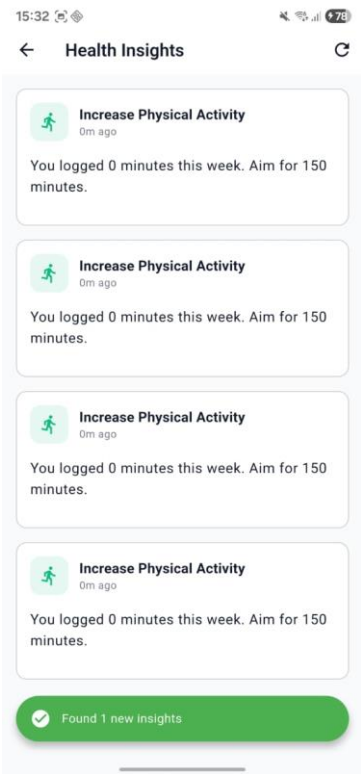
Patient Notifications



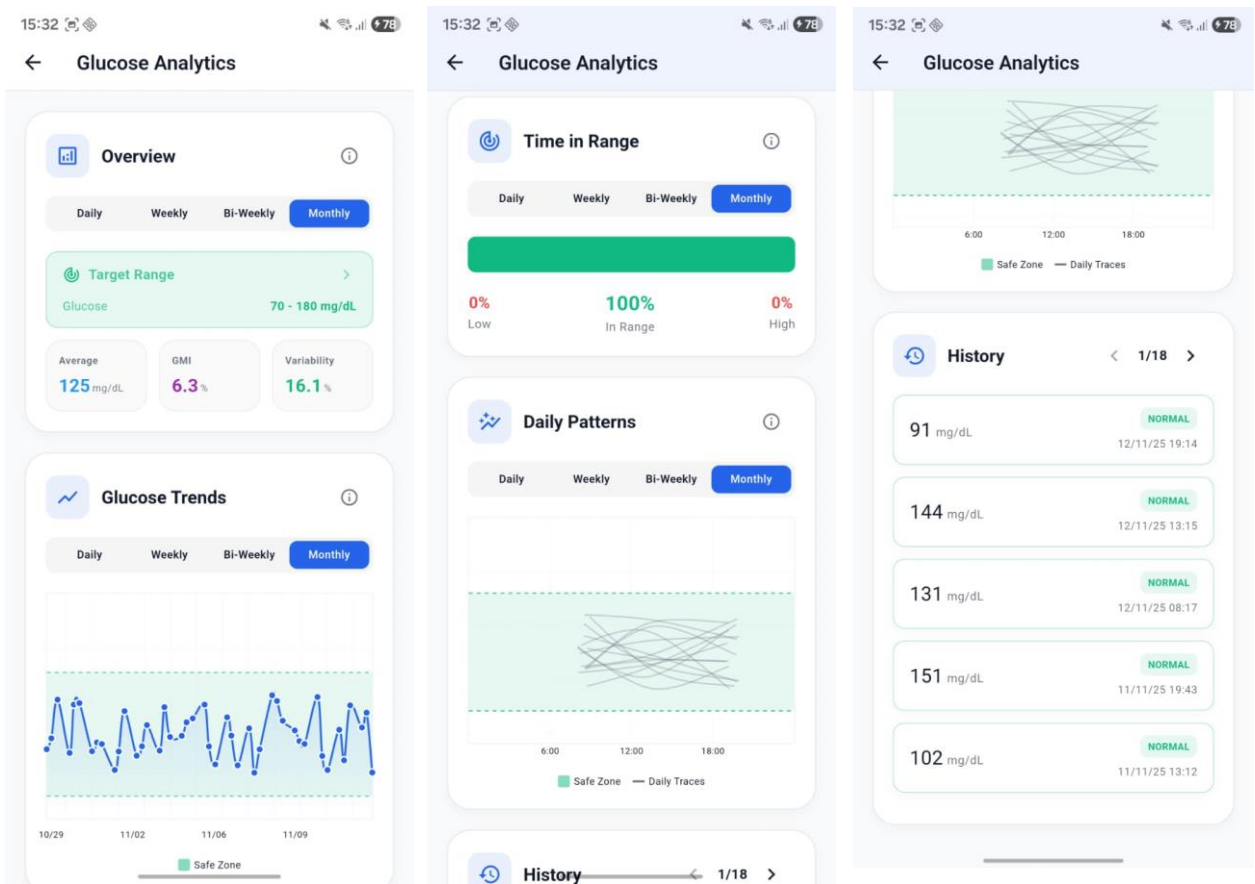
Patient Profile



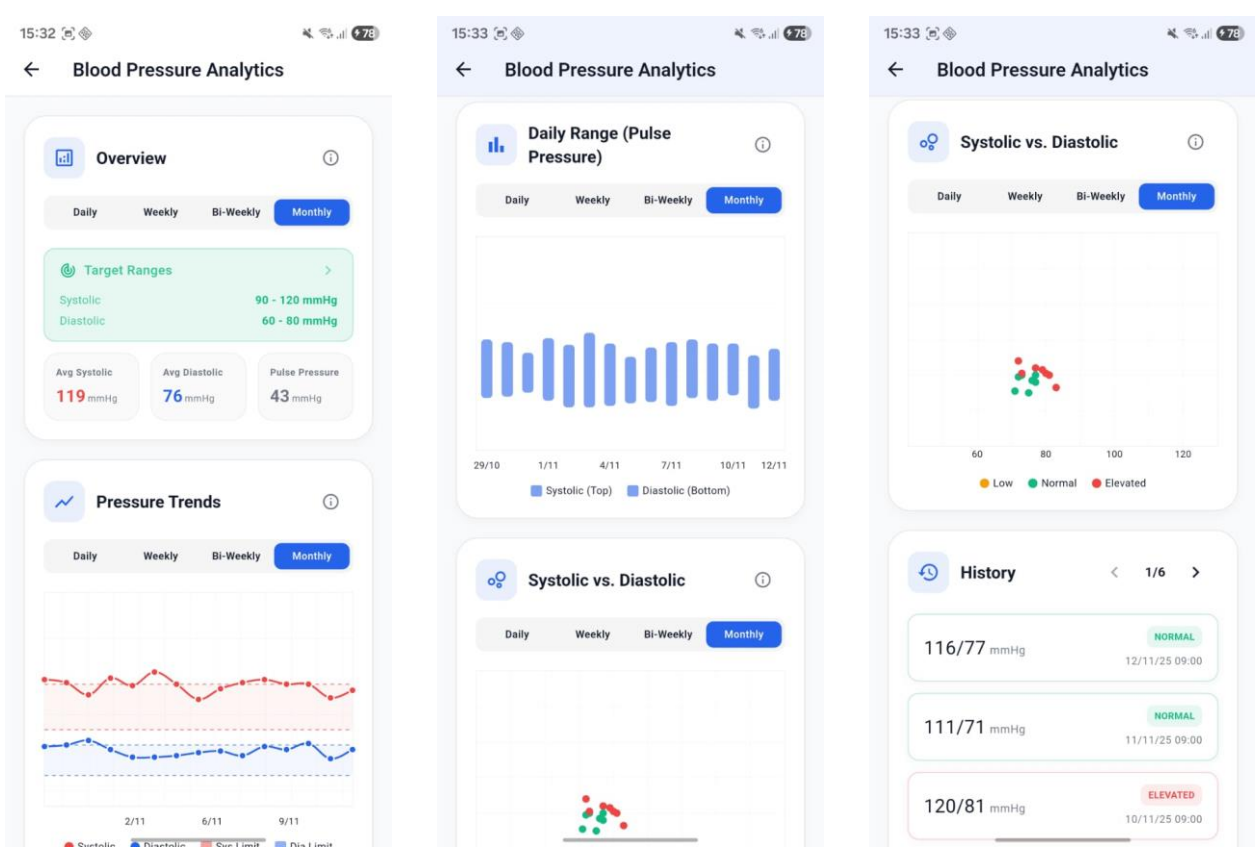
Patient Insights



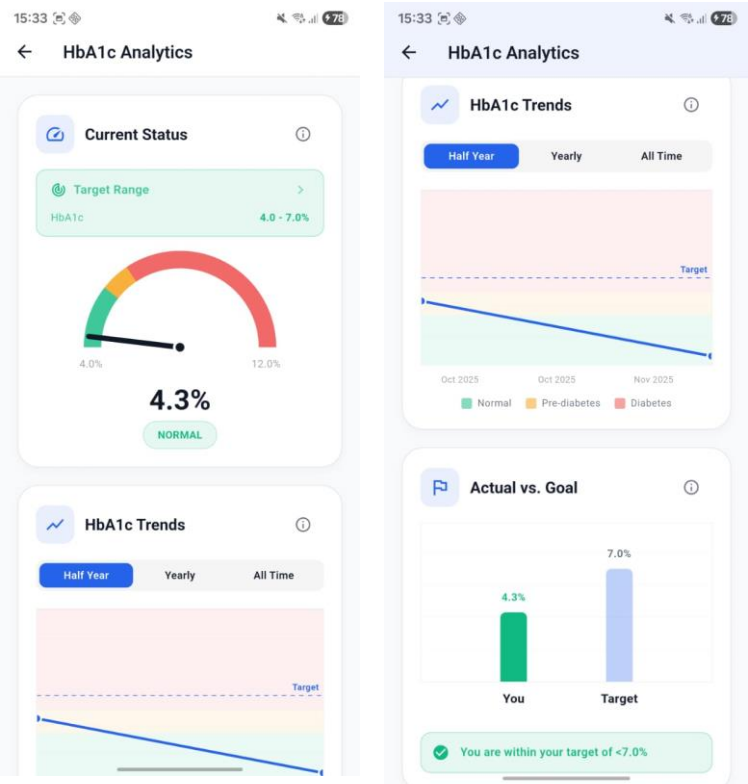
Patient Glucose Analytics



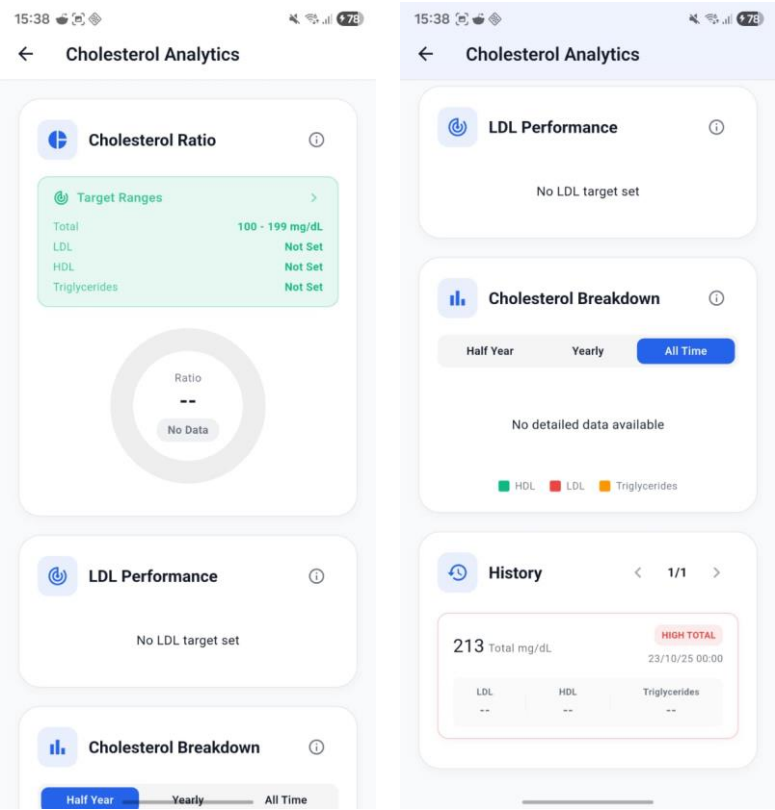
Patient Blood Pressure Analytics



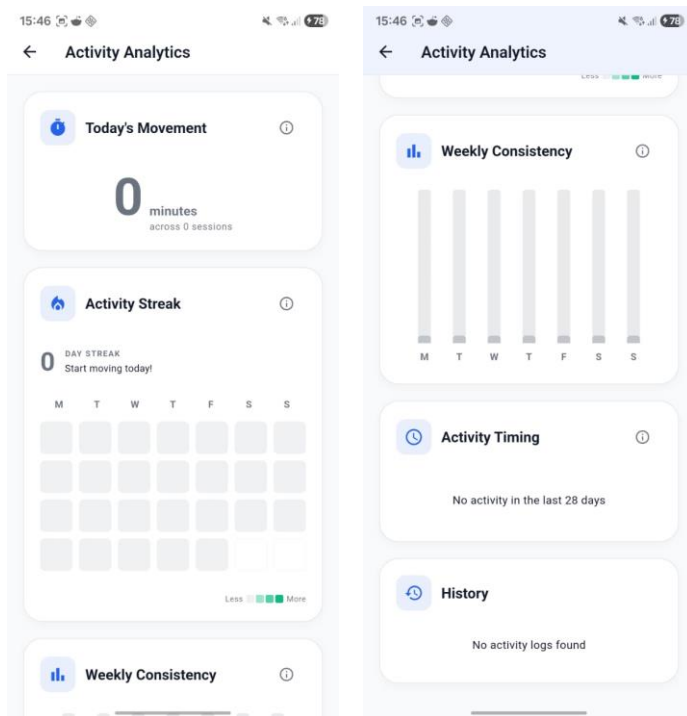
Patient HbA1c (Blood Glucose Level) Analytics



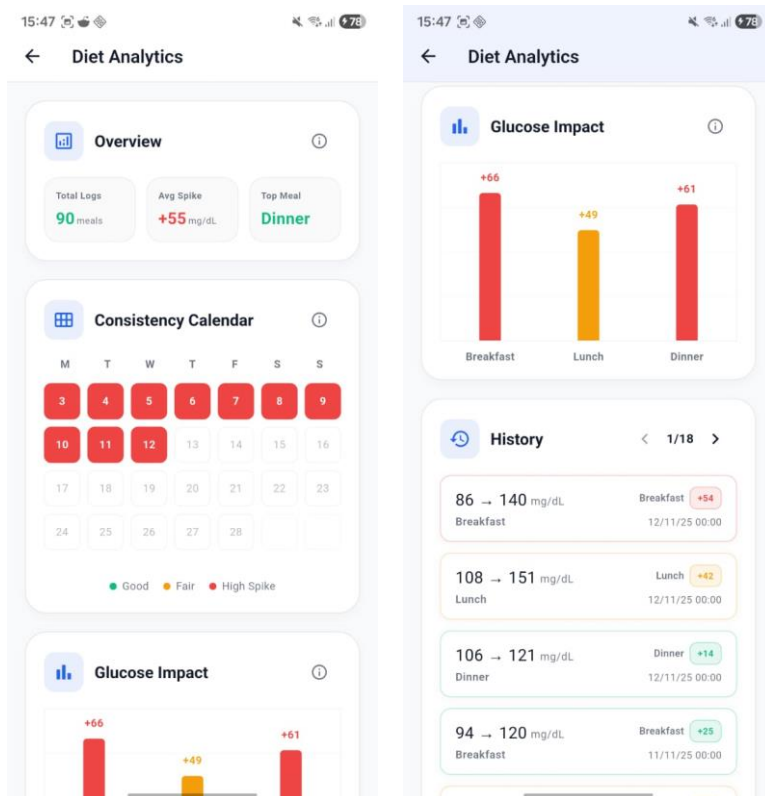
Patient Cholesterol Analytics



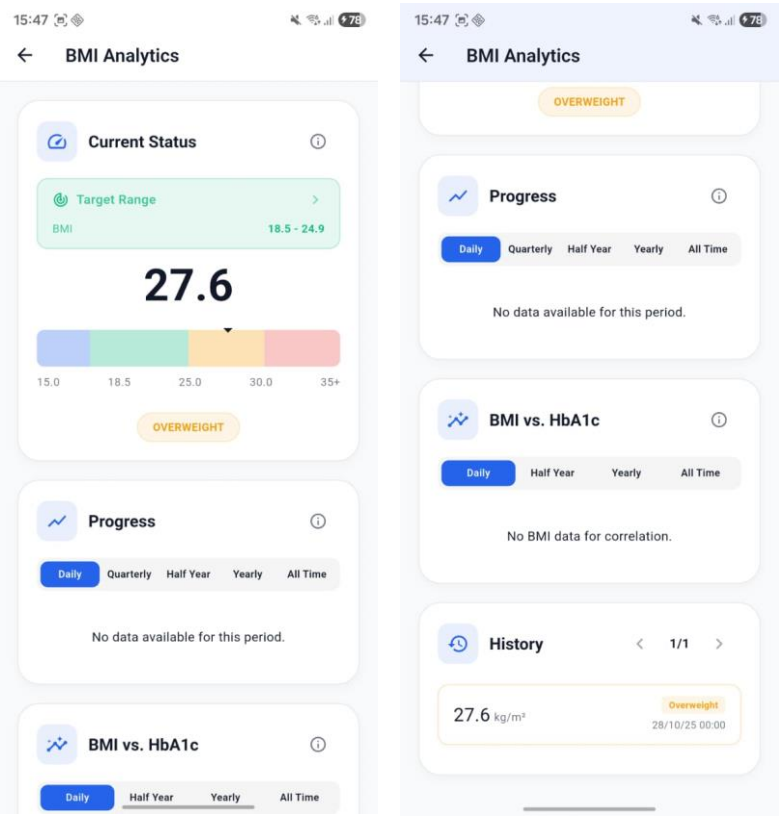
Patient Activity Analytics



Patient Diet Analytics



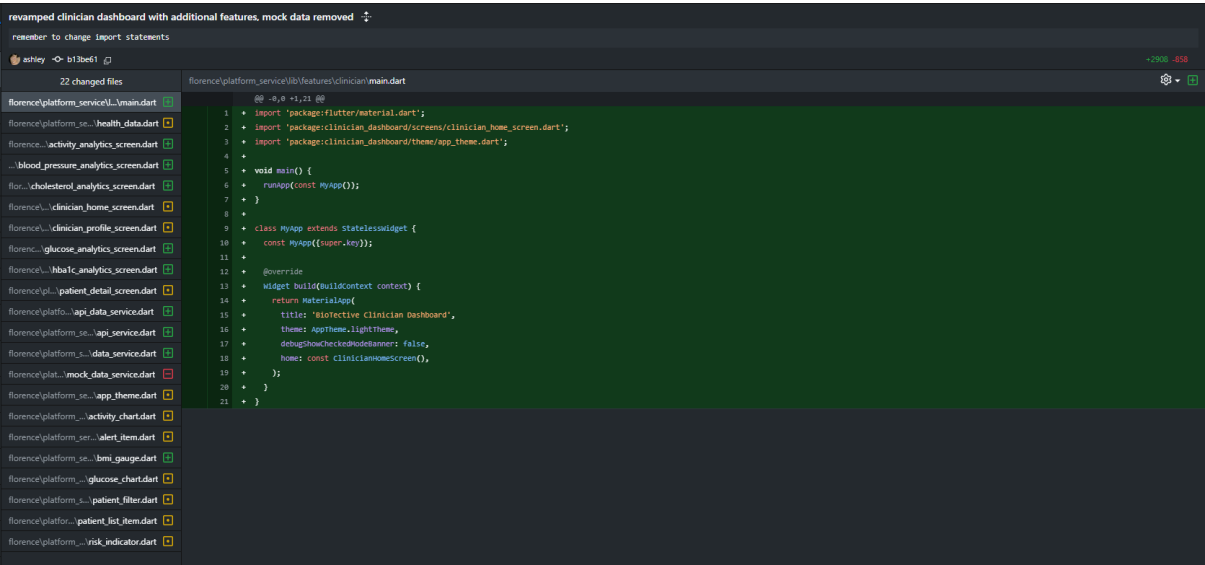
Patient BMI Analytics



2.3. Ashley (Frontend Developer - Clinician Dashboard)

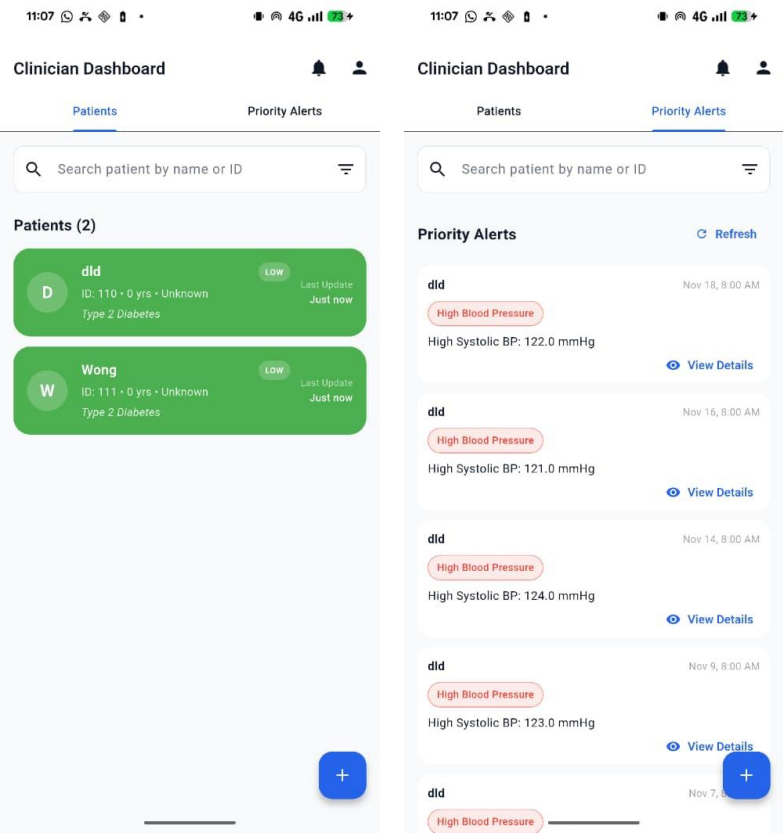
Evidence of Work

GitHub Commits

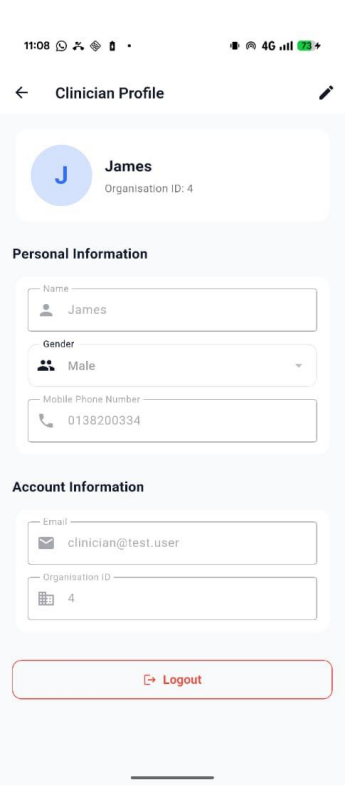


Screenshot of Application

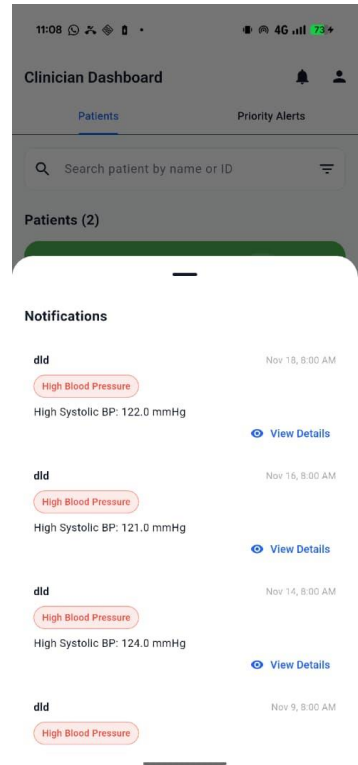
Clinician Homepage



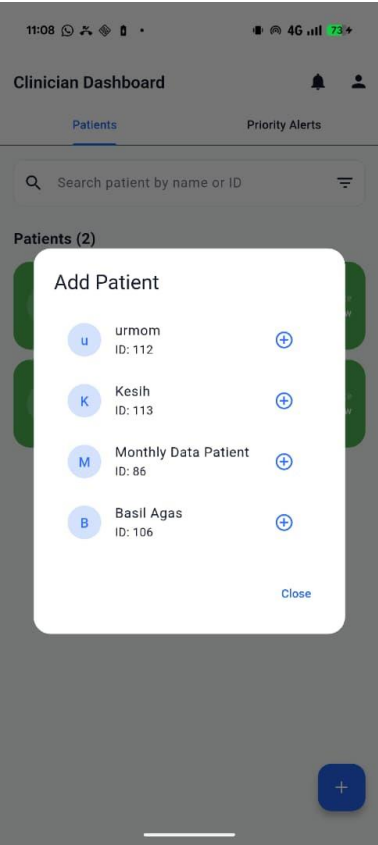
Clinician Profile Page



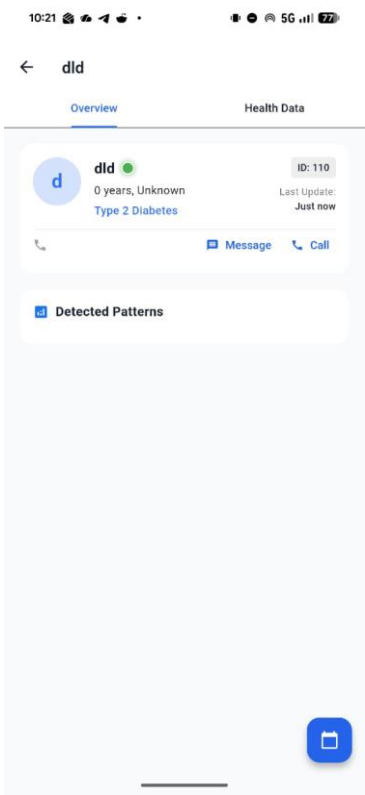
Clinician Notification



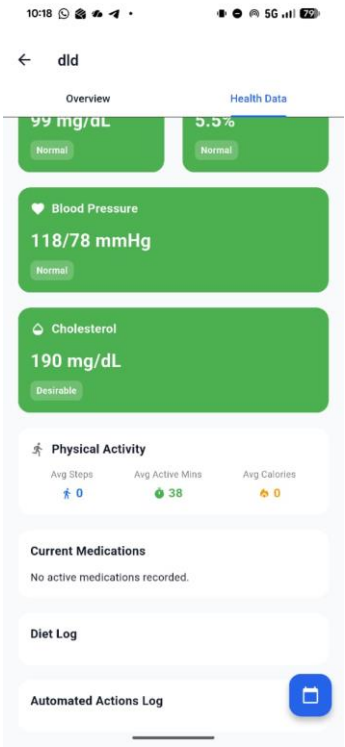
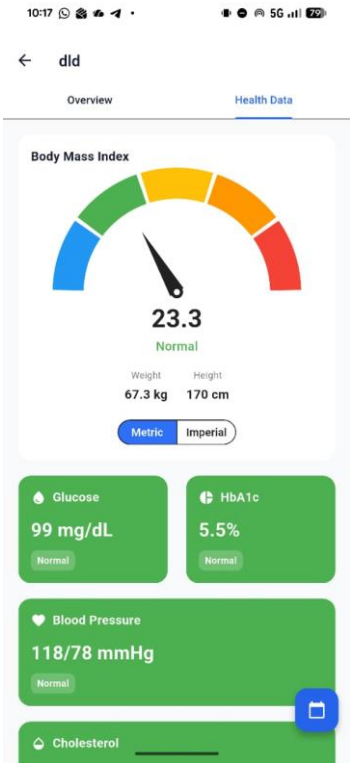
Clinician Adding Patients



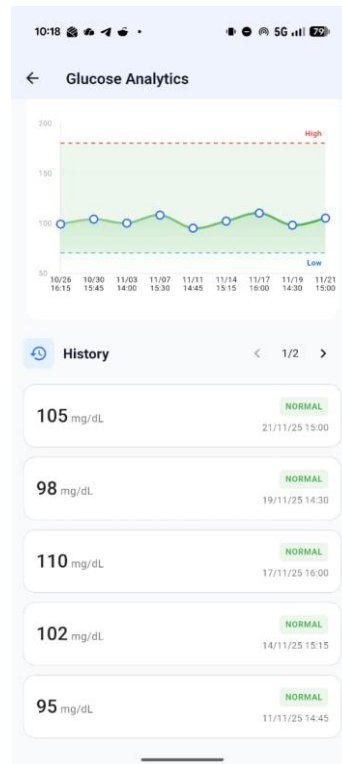
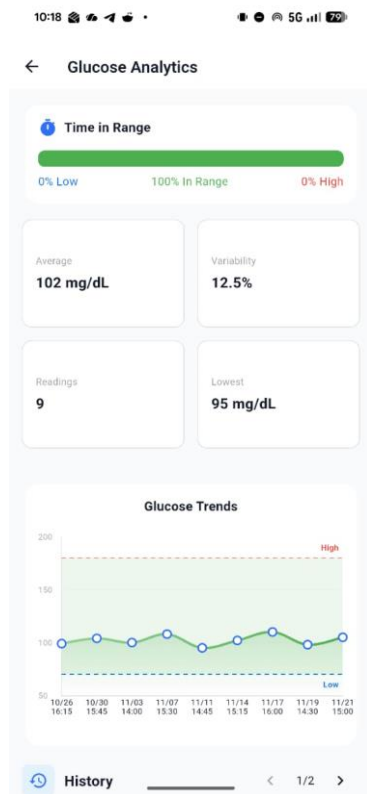
Patient's Profile Overview



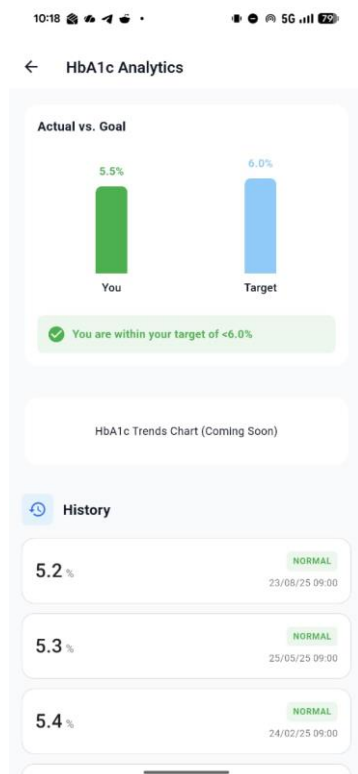
Patient's Health Data



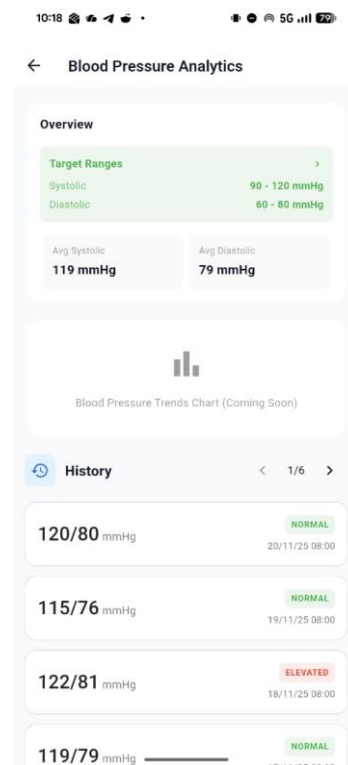
Patient's Glucose Analytics



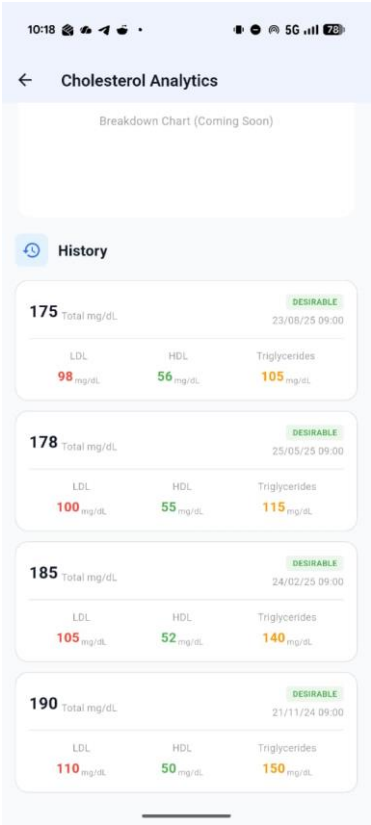
Patient's HbA1c's Analytics



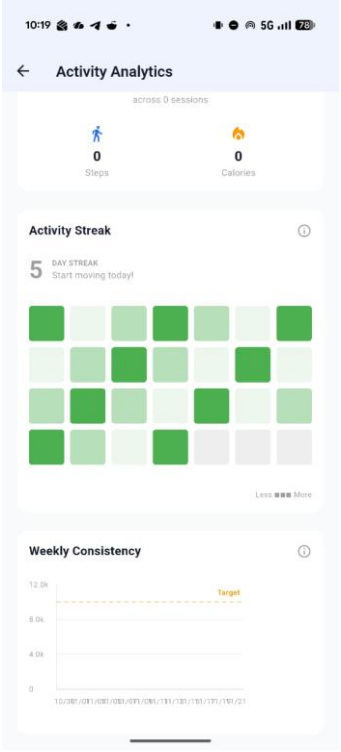
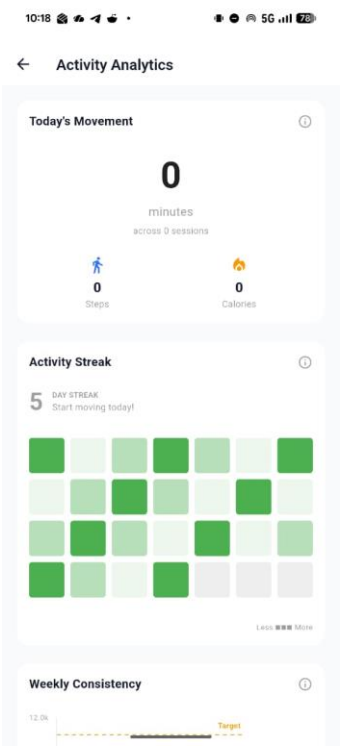
Patient's Blood Pressure Analytics



Patient's Cholesterol Analytics



Patient's Activity Analytics



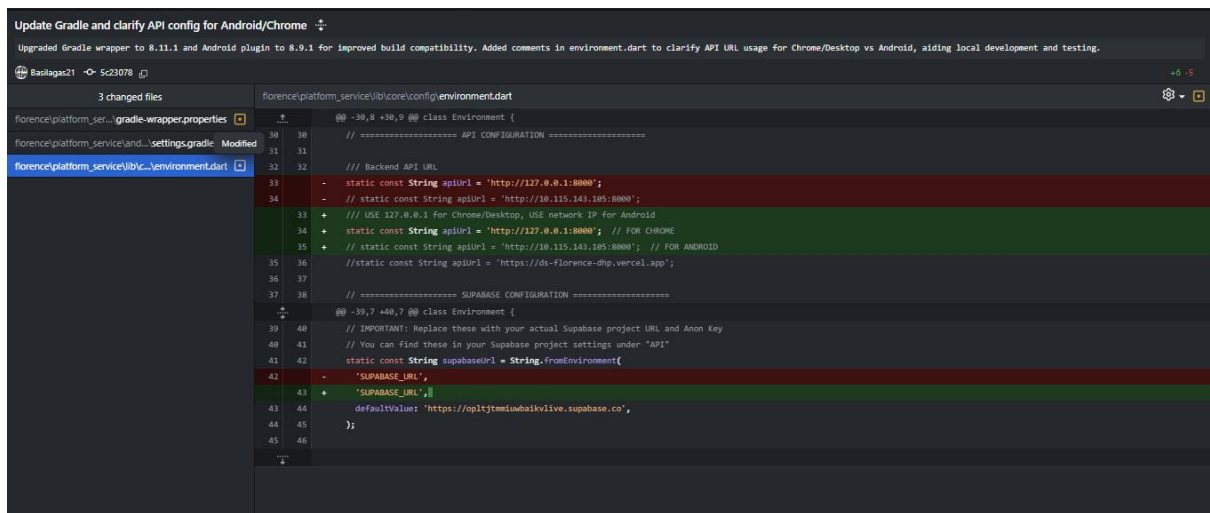
2.4. Basil (Lead AI Engineer and Data Architect)

Planned: Connect the AI functionality such as the chatbot, AI powered Recommendation, and the AI powered of the app to the working backend. Also, populate the database with patient's data

Actual: Due to the refactor of the design and 50% of the functionality of the app, I needed to make a fallback measurement and temporarily remove the AI powered Recommendation and the Pattern Recognition. Therefore, I was able to maintain the functionality of the AI chatbot where the API is securely and privately kept the API key from the Github repository. The Chatbot also now able to store the conversation of each of the patients and the patient user can choose to delete the conversation.

Evidence of Work

Github commits



```
Update Gradle and clarify API config for Android/Chrome
Upgraded Gradle wrapper to 8.11.1 and Android plugin to 8.9.1 for improved build compatibility. Added comments in environment.dart to clarify API URL usage for Chrome/Desktop vs Android, aiding local development and testing.

3 changed files
fiorcelce/platform_ser.../gradle-wrapper.properties
fiorcelce/platform_ser.../settings.gradle
fiorcelce/platform_ser.../environment.dart

@@ -30,8 +30,9 @@ class Environment {
// ===== API CONFIGURATION =====
// Backend API URL
- static const String apiUrl = 'http://127.0.0.1:8080';
- // static const String apiUrl = 'http://10.115.163.105:8080';
+ // USE 127.0.0.1 for Chrome/Desktop, USE network IP for Android
+ static const String apiUrl = 'http://127.0.0.1:8080'; // FOR CHROME
+ // static const String apiUrl = 'http://10.115.163.105:8080'; // FOR ANDROID
//static const String apiUrl = 'https://ds-florence-dhp.vercel.app';

// ===== SUPABASE CONFIGURATION =====
@@ -39,7 +40,7 @@ class Environment {
// IMPORTANT! Replace these with your actual Supabase project URL and Anon Key
// You can find these in your Supabase project settings under "API"
static const String supabaseUrl = String.fromEnvironment(
- 'SUPABASE_URL',
+ 'SUPABASE_URL',
defaultValue: 'https://optjtmuwbalkivlive.supabase.co',
);
```


Add Florence Chatbot microservice core files

Initial implementation of the Florence Chatbot microservice, including FastAPI app setup, configuration, authentication utilities, health data aggregation, DeepSeek LLM integration, conversation history management, API endpoints, and Pydantic models. Enables multi-tenant, secure, AI-powered health chatbot functionality decoupled from the frontend.

basilagas21 9d30b37

+1871

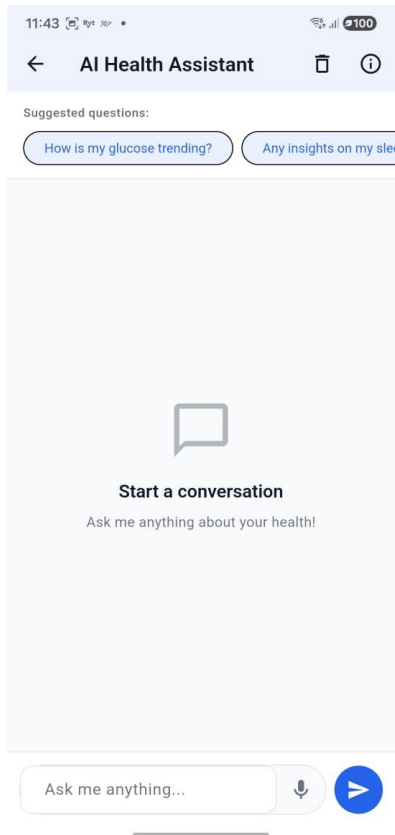
- 15 changed files
- florence/chatbot_service/README.md
- florence/chatbot_service/config.py
- florence/chatbot_service/main.py
- florence/chatbot_service/models/_init_.py
- florence/chatbot_service/models/chat.py
- florence/chatbot_service/models/health.py
- florence/chatbot_service/requirements.txt
- florence/chatbot_service/routers/_init_.py
- florence/chatbot_service/routers/chat.py
- florence/chatbot_service/services/_init_.py
- florence/chatbot_service/services/conversation.py
- florence/chatbot_service/services/deepseek.py
- florence/chatbot_service/services/health_data.py
- florence/chatbot_service/utils/_init_.py
- florence/chatbot_service/utils/auth.py

florence/chatbot_service/README.md

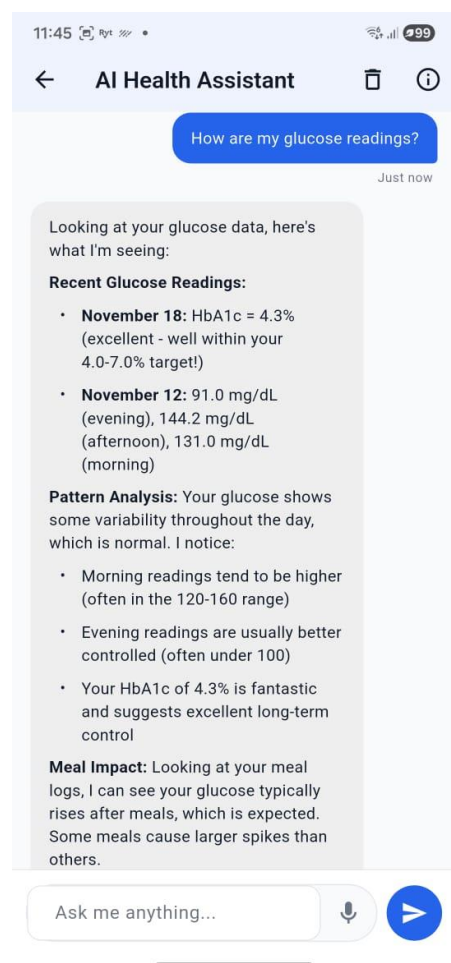
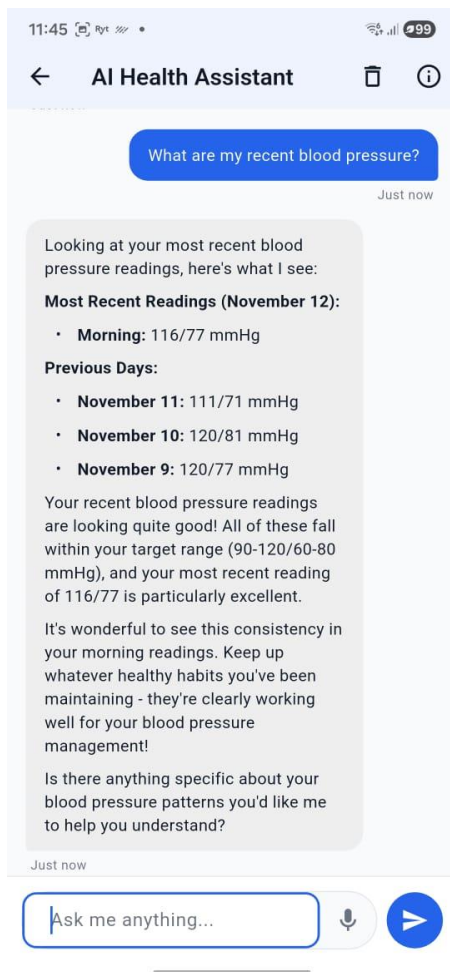
```
@@ -0,0 +1,414 @@
1 + # Florence Chatbot Microservice
2 +
3 + A dedicated multi-tenant Python microservice for AI-powered health chatbot functionality, decoupled from the Flutter frontend.
4 +
5 + ## Overview
6 +
7 + This service provides AI-powered conversational assistance for diabetes management by:
8 + - Managing chat logic and conversation history
9 + - Integrating with DeepSeek LLM for intelligent responses
10 + - Fetching and aggregating patient health data
11 + - Ensuring strict user data isolation and security
12 +
13 + ## Architecture
14 + ...
15 +
16 + Flutter App + JWT Auth + Chatbot Service + Supabase Database
17 + |
18 + | DeepSeek API
19 + ...
20 +
21 + ## Key Principles
22 +
23 + 1. **Multi-Tenant**: Handles multiple users concurrently with strict data isolation
24 + 2. **Stateless**: All conversation history stored in database
25 + 3. **Secure**: JWT authentication on every request
26 + 4. **Scalable**: Independent deployment and horizontal scaling
27 +
28 + ## Project Structure
29 + ...
30 + ...
31 + chatbot_service/
32 + |--- main.py           # FastAPI application entry point
33 + |--- config.py        # Environment configuration
34 + |--- requirements.txt  # Python dependencies
35 + |--- database_schema.sql # Supabase table schema and RLS policies
36 + |--- README.md        # This file
37 + |--- routers/
38 + |   |--- _init_.py
39 + |   |--- chat.py      # Chat API endpoints
40 + |--- services/
41 + |   |--- _init_.py
42 + |   |--- deepseek.py  # DeepSeek API integration
43 + |   |--- health_data.py # Health data aggregation
44 + |   |--- conversation.py # Chat history management
45 + |--- models/
46 + |   |--- _init_.py
```

Screenshots of the Application

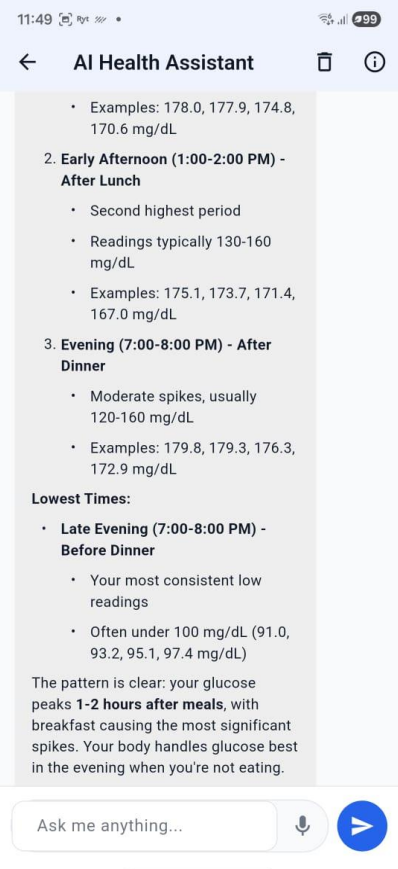
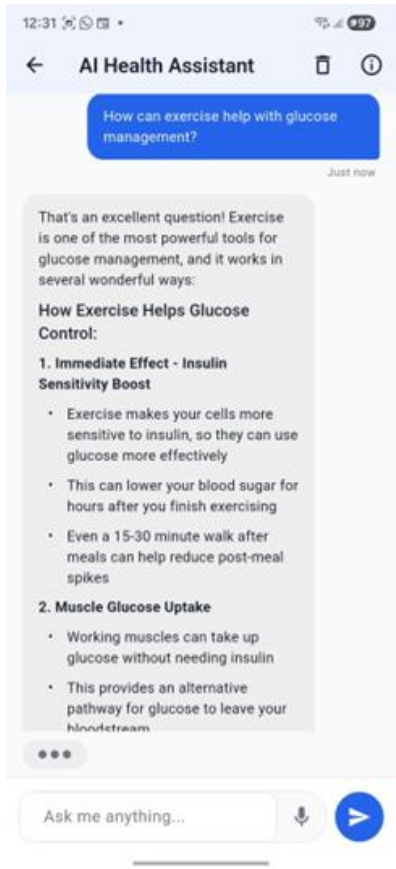
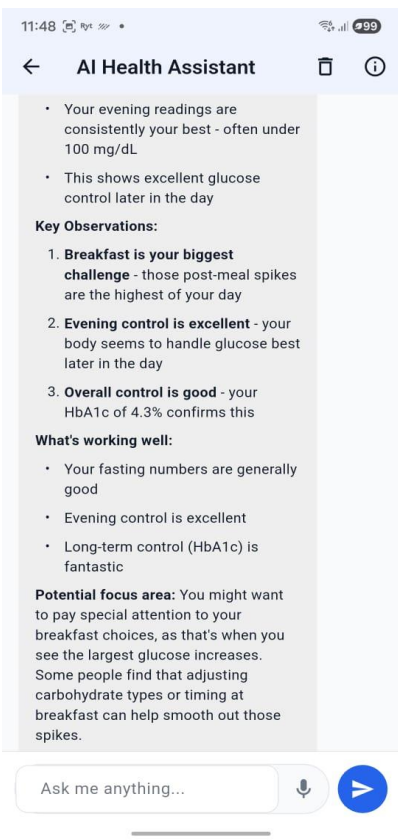
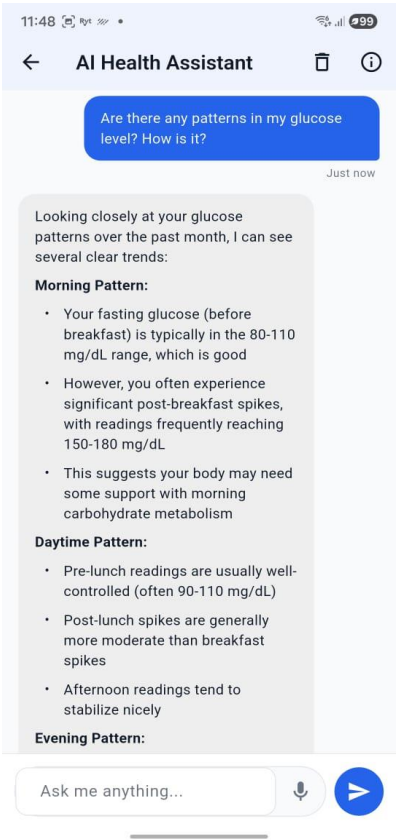
AI Chatbot UI Page (Default)



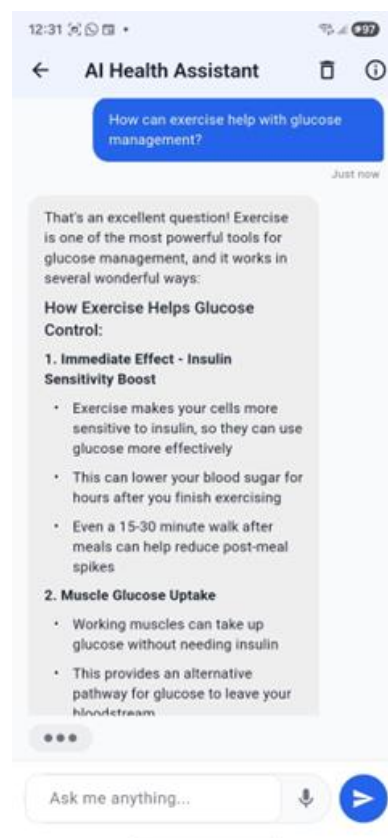
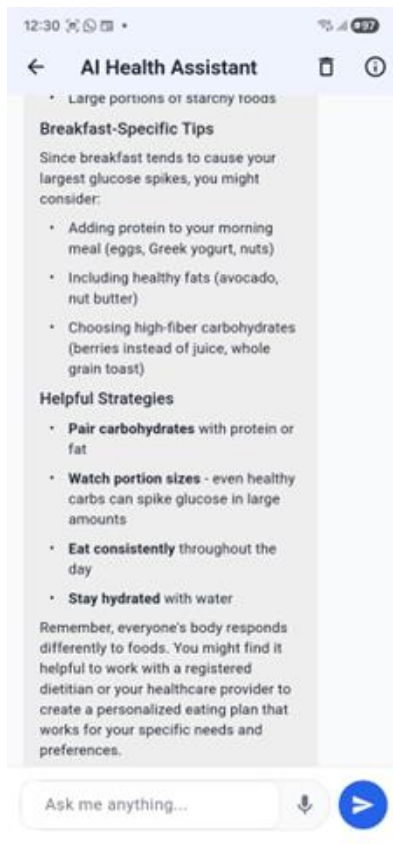
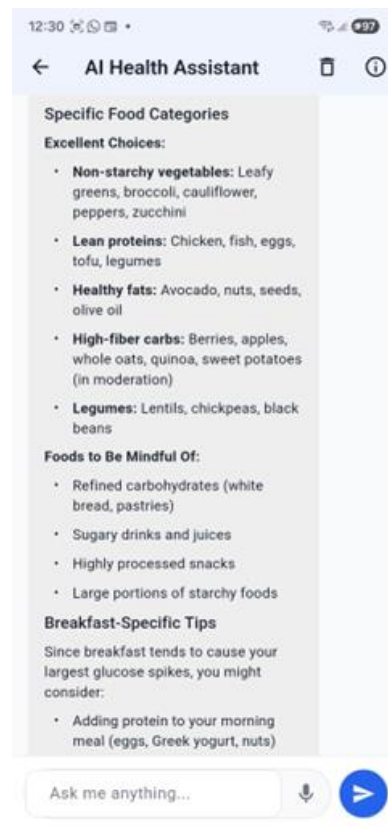
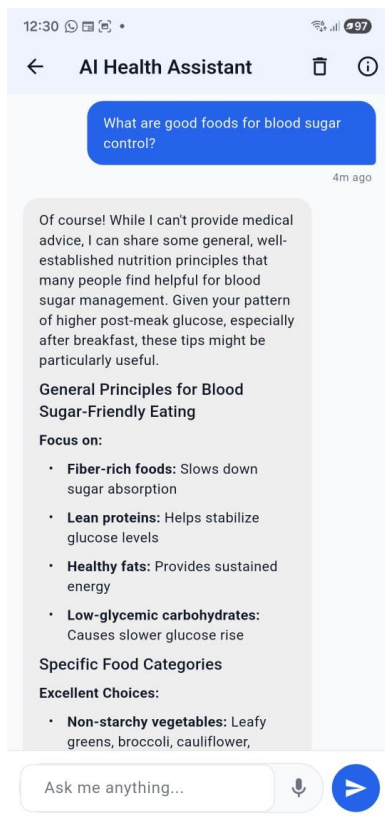
AI Chatbot Test (Health Data)

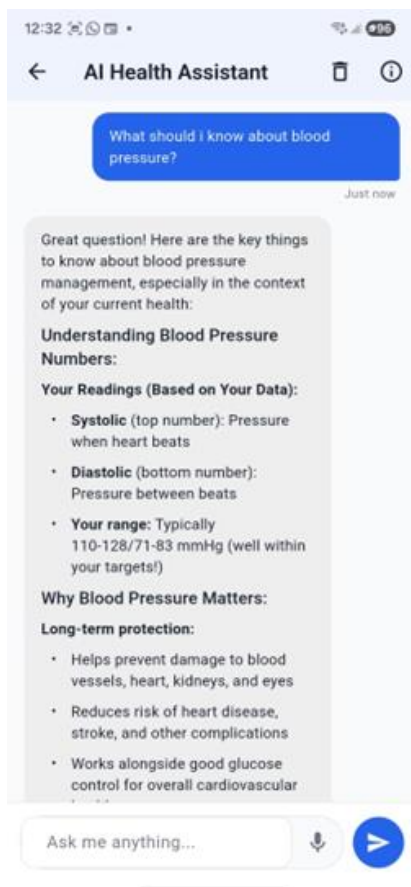
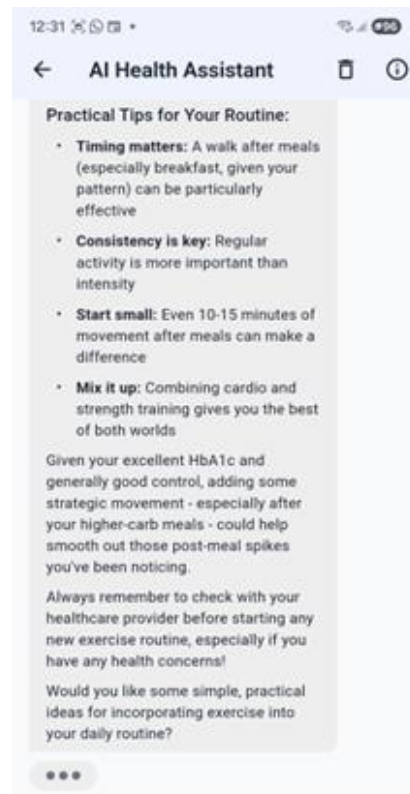
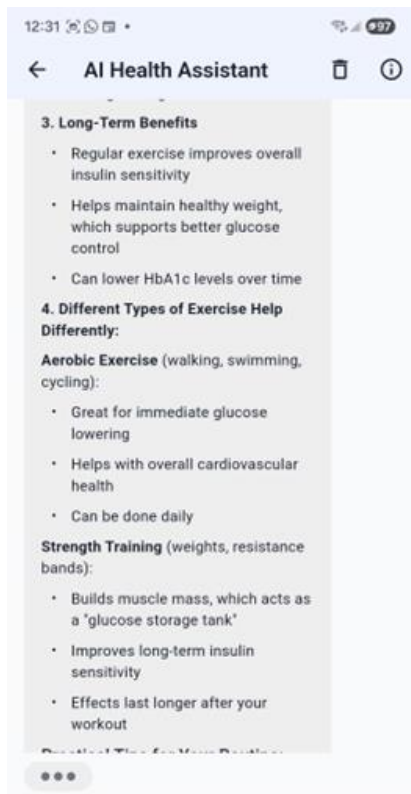


AI Chatbot Test (Patterns and Trends)

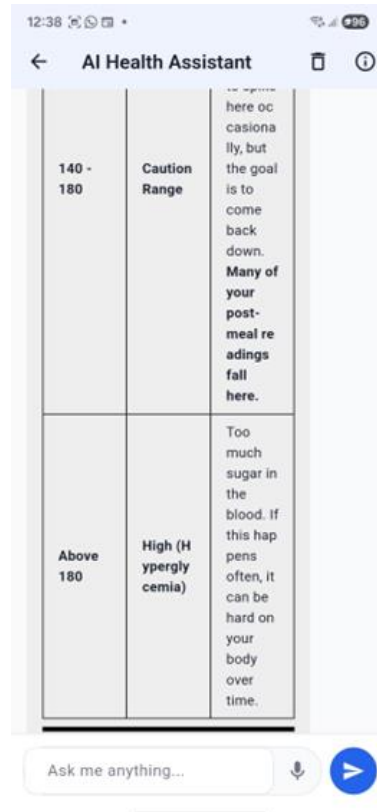
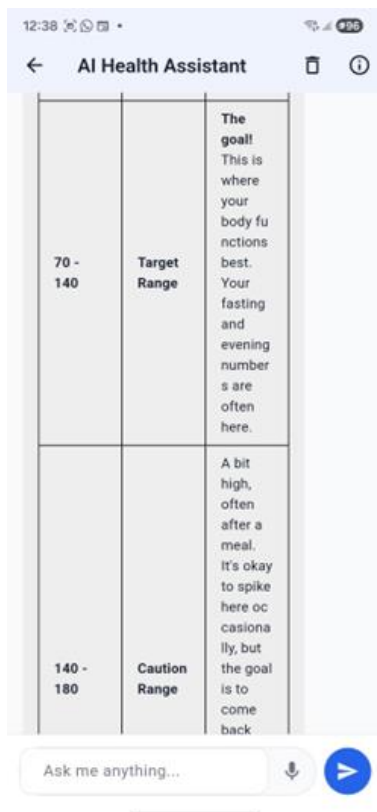
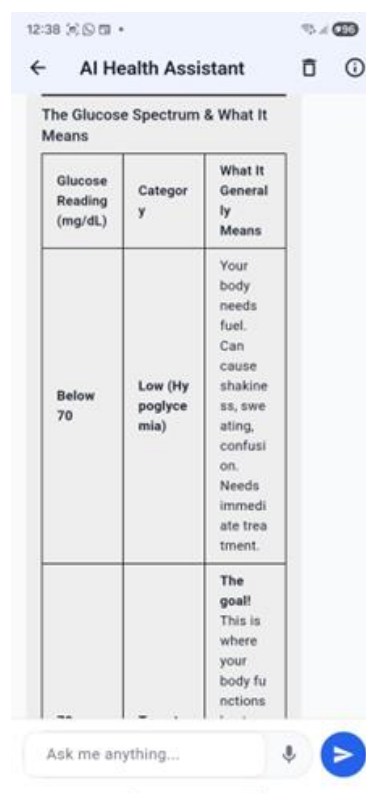
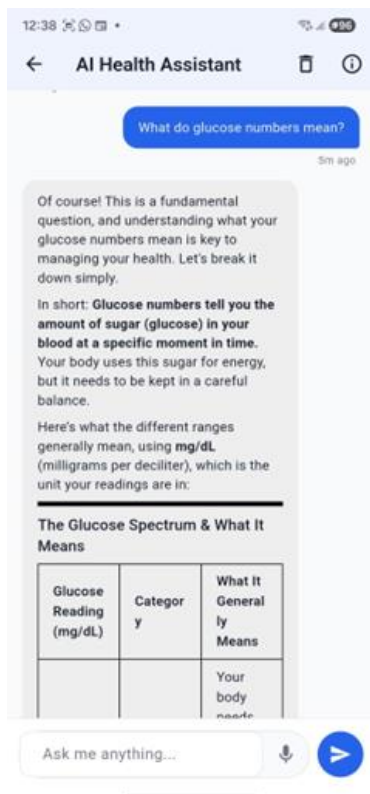


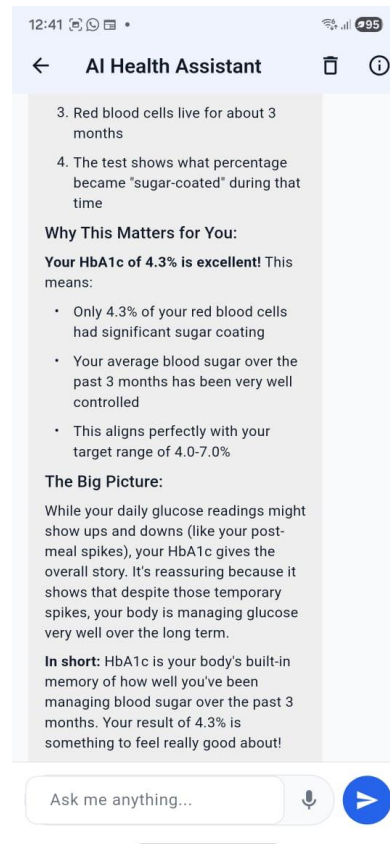
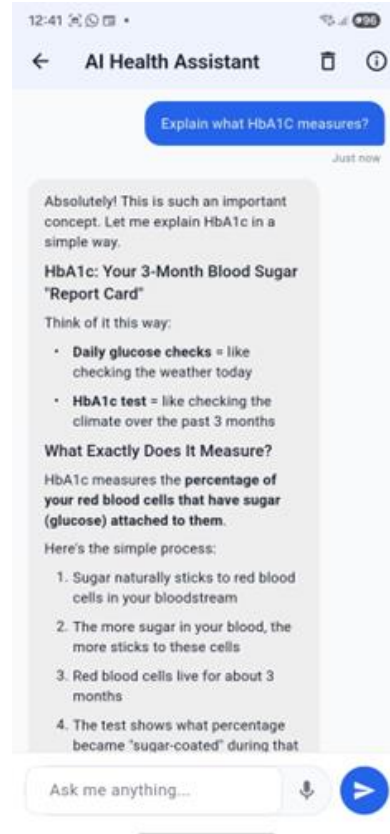
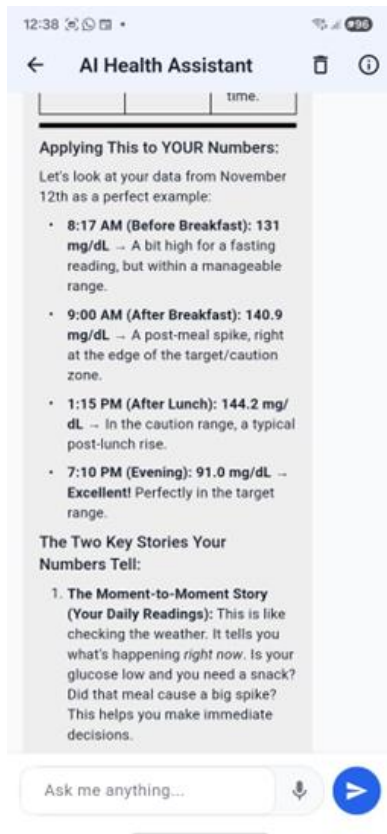
AI Chatbot Test (General Health Questions)



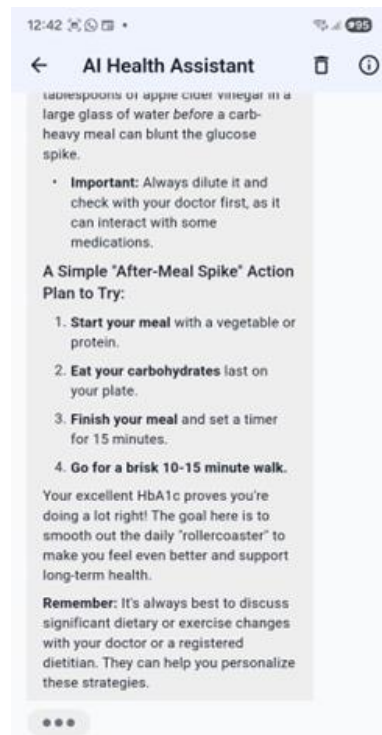
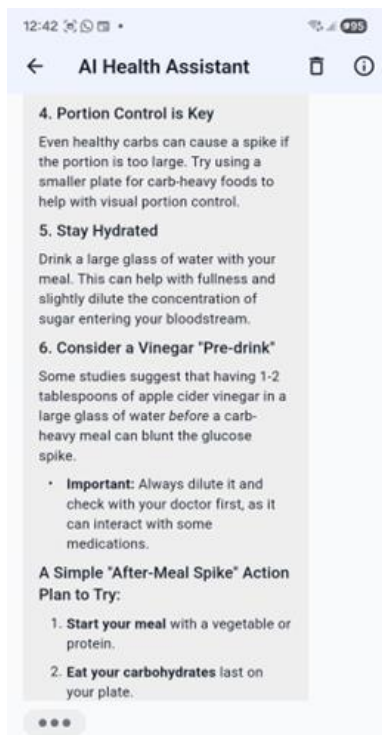
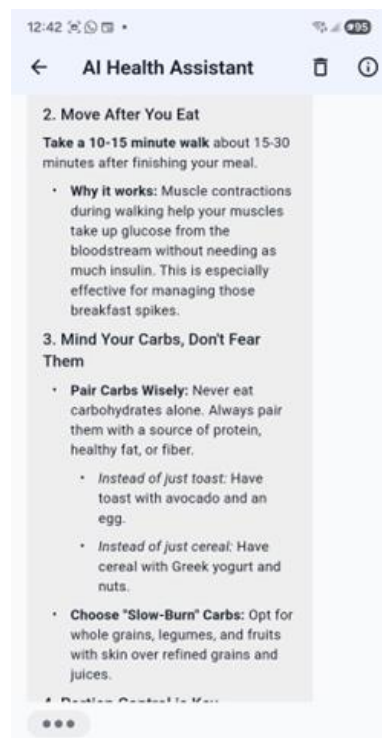
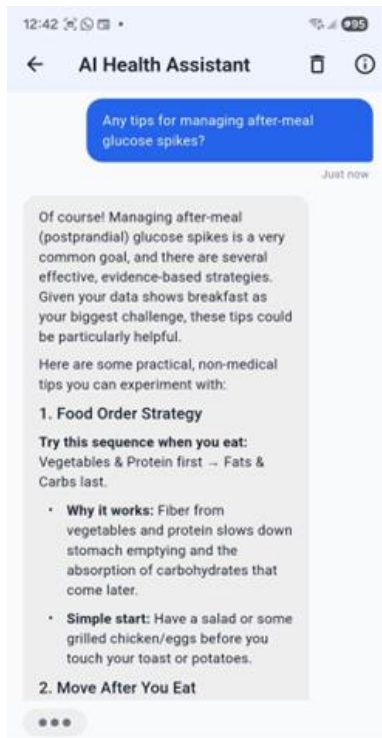


AI Chatbot Test (Understanding the metrics/number/measurement)





AI Chatbot Test (Daily Management)



AI Chatbot with the Working Backend

Filter		Sort		Insert		6 RLS policies		Enable Realtime		Role postgres		DB	
	i. u... v	patient... i... v	role text v	content text v	timestamp timestamptz v	cont							
	0377b5f3-68c3	86	→	assistant	Of course! While I can't provide medical advice, I can sh	2025-11-28 04:25:50.627068+ {"da							
	039ba38b-fa16	86	→	assistant	Looking at your most recent blood pressure readings, he	2025-11-28 03:44:58.300033+ {"da							
	07b660f8-faba	86	→	user	How can exercise help with glucose management?	2025-11-28 04:30:58.342194+ NUL							
	2312f258-2e7a	110	→	user	Hi, nice to meet you	2025-11-28 17:06:09.959378+ NUL							
	259246ee-ed18	86	→	user	Are there any patterns in my glucose level? How is it?	2025-11-28 03:47:58.108013+ NUL							
	52bf1e97-8505	86	→	assistant	Looking at your glucose patterns, here are when your le	2025-11-28 03:49:42.18248+ {"da							
	5b18e78e-3c74	86	→	user	What do glucose numbers mean?	2025-11-28 04:32:43.50239+ NUL							
	5dd63045-bdca	86	→	user	What are my recent blood pressure?	2025-11-28 03:44:44.357903+ NUL							
	631250dd-16bb	86	→	user	What times of the day are the highest?	2025-11-28 03:49:21.710464+ NUL							
	70421642-7d86	86	→	user	Any tips for managing after-meal glucose spikes?	2025-11-28 04:41:45.579077+ NUL							
	795ebb0f-996a	86	→	assistant	Great question! Here are the key things to know about b	2025-11-28 04:31:51.079132+ {"da							
	7bf418c8-cae2	110	→	user	1. Please address me by my name in this conversation. 2.	2025-11-28 17:07:07.000612+ NUL							

2.5. Edison (Backend Developer)

My contribution this sprint bridged the gap between advanced backend AI logic and the frontend user experience. I worked in close collaboration with **Daniel** to design and implement the Data Visualisation dashboard.

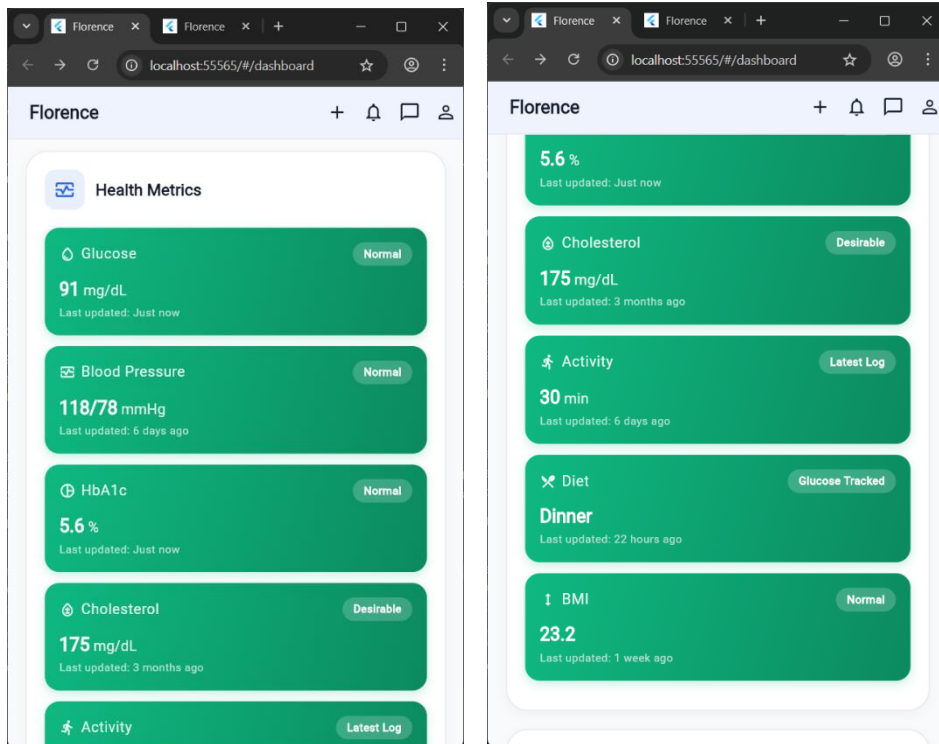
Frontend Data Visualisation:

On the client side, I worked alongside **Daniel** to implement the comprehensive **Health Dashboard and Data Visualisation suite**.

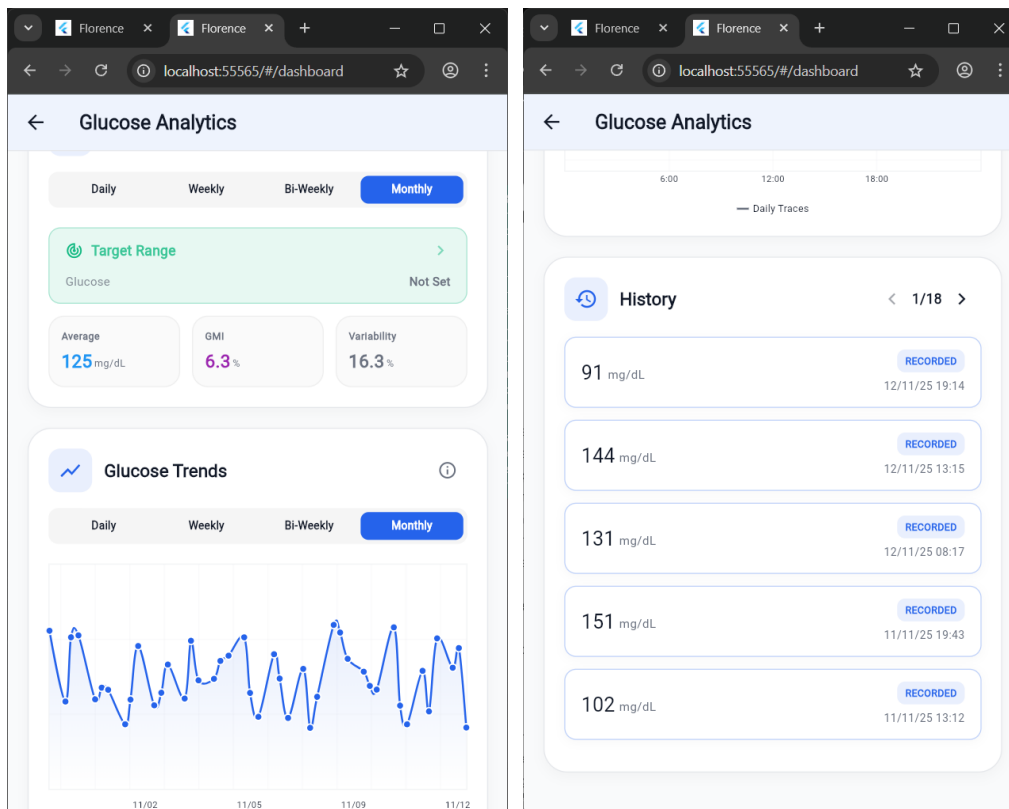
- **Interactive Analytics:** We co-developed the detailed analytics screens for Glucose, Blood Pressure, and HbA1c. I utilised `fl_chart` to build complex visualisations, such as dynamic line charts for trends and scatter plots for correlation analysis.
- **Biometric Cards:** I implemented the reusable `CompactHealthCard` and `BiometricsSection` widgets to summarise critical health metrics, while collaborating with **Daniel** to ensure the state management (via `Riverpod`) correctly updated these UI components in real-time.

Evidence of Work

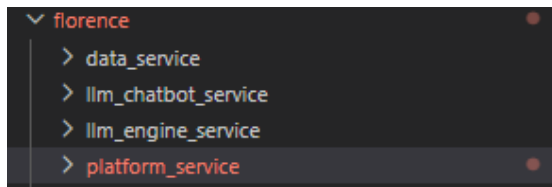
The “Biometrics” Dashboard (Frontend)



Deep Dive Analytics Chart (Frontend)



The Microservice Architecture (File Structure)



Context Injection Logic (Code)

```
186 # Singleton instance
187 _health_data_service: Optional[HealthDataService] = None
188
189
190 def get_health_data_service() -> HealthDataService:
191     """Get or create the singleton HealthDataService instance."""
192     global _health_data_service
193     if _health_data_service is None:
194         _health_data_service = HealthDataService()
195     return _health_data_service
196
```

3. Sprint Plan

This sprint focused on **EPIC 3: Integration and Intelligence**, covering Milestone 5 and parts of 6. Following the architectural refactoring in Sprint 2, the primary goal was to operationalise the Data Service as the central source of truth and fully implement the patient-facing AI Chatbot.

A critical strategic decision was made during Sprint Planning to **prioritise the conversational Chatbot and Data Visualisation** over the background Automation Engine (LAM Triggers). This decision was based on the client's preference for a tangible, interactive user experience for the prototype demonstration.

Sprint Backlog

Story ID	User Story	Priority	Story Points	Status
T3.1	As a patient, I want to chat with an AI assistant that remembers my previous context so that I can have a natural conversation about my health.	High	13	Completed
T2.2	As a patient, I want to see my detailed Glucose, BP and BMI data from the backend displayed as interactive charts populated with real data from the backend.	High	8	Completed
T1.3	As a user, I want a robust authentication system (Email/Password) with deep-linking support for password resets and verification.	High	8	Completed
T5.1	As a clinician, I want my dashboard to fetch real patient lists and see priority patients.	High	13	Completed
T6.1	As a developer, I want to implement a dedicated Microservice for the Chatbot	Medium	8	Completed

	to handle LLM context and state management separately from the core data API.			
T4.1	<i>[DEFERRED]</i> As a developer, I want to implement the background LLM Automation Engine for proactive triggers.	Low	13	Moved to Next Semester

Table 1: Sprint 3 Backlog showing completed core features and the strategic deferral of the automation engine.

4. Sprint Progress

The team achieved significant velocity this sprint, successfully integrating the frontend applications with the backend services. The system has moved from a "mock-up" phase to a fully functional "connected" prototype.

4.1. The Data Service (Central Hub)

We successfully finalised the **Florence Data Service** (FastAPI). It now acts as the single source of truth, handling all CRUD operations for patients, clinicians and health logs.

- I. **Security and Access Control:** We implemented a comprehensive Row-Level Security (RLS) architecture on Supabase to enforce strict data isolation across three distinct user roles:
 - a. **Patients:** Policies utilise *auth.uid()* to ensure patients have exclusive Read/Write access only to records directly linked to their own *user_id* (e.g., *patient_monitor_data*, *patient_chat_history*).
 - b. **Clinicians:** Implemented relational policies that grant read access to patient data only if the *patient_profiles.clinician_id* matches the authenticated clinician's ID. This ensures that clinicians can only view data for patients who are explicitly assigned to them.
 - c. **Admins:** Implemented Role-Based Access Control (RBAC), examining the JWT *app_metadata*. Users with the *ADMIN* role claim are granted global management permissions across all tables (profiles, logs and organisations) for system oversight.
- II. **API Development:** Developed a rich suite of RESTful endpoints organised by functional domain to support distinct user workflows:
 - a. **Patient Self-Management:** Implemented "me"-scoped endpoints allowing patients to log and retrieve diverse health metrics (Glucose, BP, HbA1c, BMI), track lifestyle data (Meals, Activity) and view their personalised health thresholds.
 - b. **Clinician Monitoring:** Created aggregation endpoints that allow clinicians to fetch their assigned patient roster, view detailed patient histories, access computed priority alerts and append clinical notes.

- c. **System Administration:** Developed high-level management endpoints for onboarding users, configuring global settings and managing patient-clinician assignments.

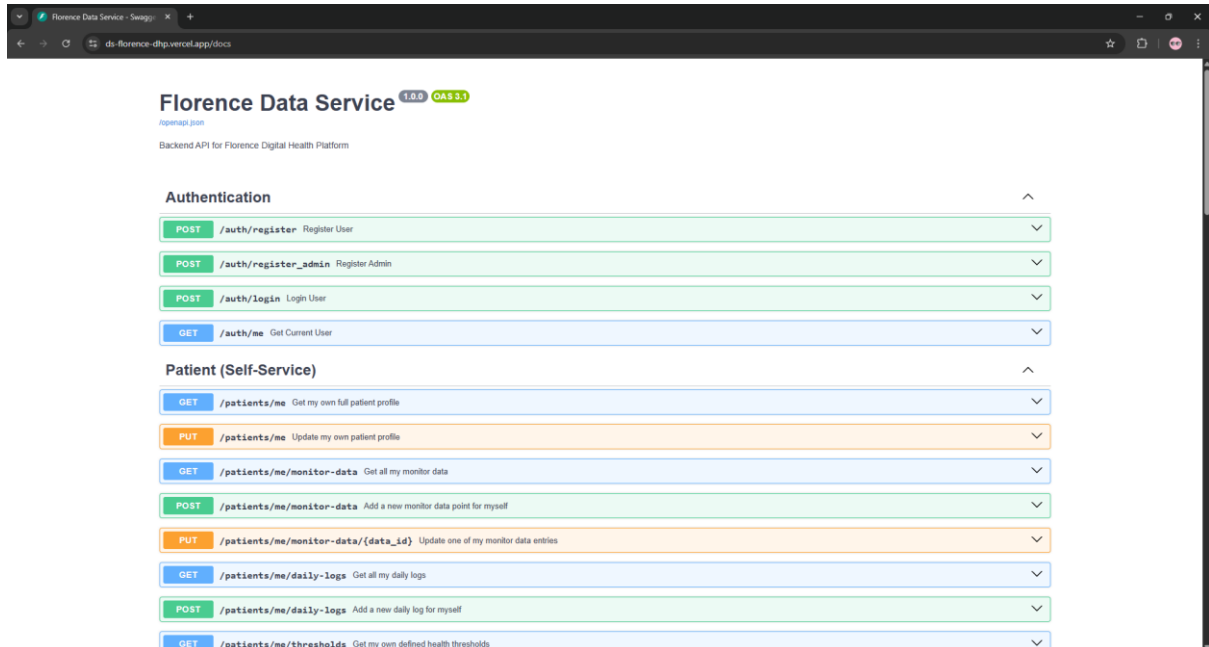


Figure 1: Swagger UI/OpenAPI documentation showing the full list of active endpoints

4.2. Patient Dashboard and Visualisation

The Patient App underwent a major architectural and visual overhaul to improve usability and data clarity.

- I. **Structural Refinement:** We redesigned the dashboard to enforce a clear separation of concerns. AI-driven features were consolidated into a dedicated "AI Insight Card", while standard vital signs were organised into a distinct "Health Metrics" section. This allows users to instinctively differentiate between raw health data and AI-generated guidance.
- II. **Live Data Integration:** Replaced static placeholders with the *fl_chart* library, fully integrated with our *HealthDataProvider*. Charts now render real-time historical data fetched directly from the Data Service.
- III. **Dynamic Visual Logic:** Implemented threshold-aware rendering logic. Visual elements automatically shift colors (Green/Amber/Red) based on the patient's specific health targets (e.g., a glucose reading > 180 mg/dL instantly turns the data point red), providing immediate, at-a-glance health context.
- IV. **Targeted Visualisation Strategy:** We conducted research to select the most appropriate visualisation type for each specific metric (e.g., using Scatter Charts for Blood Pressure correlation vs. Line Charts for Glucose trends), ensuring the analytics pages provide maximum readability and clinical value.

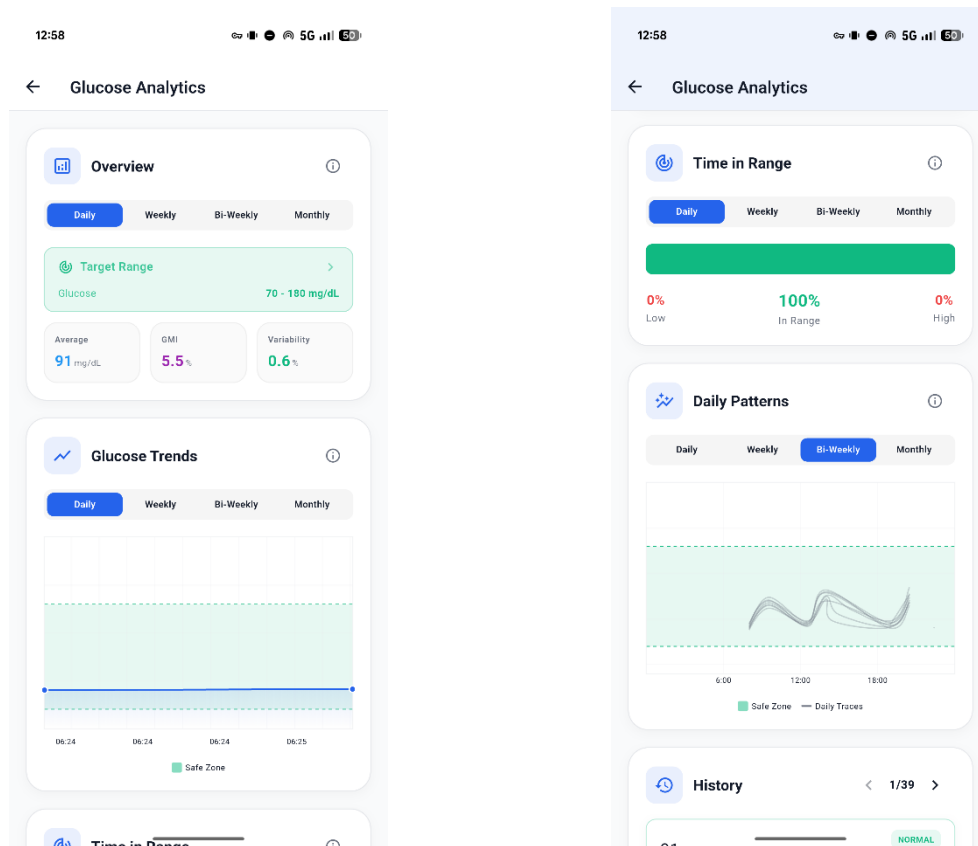


Figure 2: The Glucose Analytics screen showing the graph populated with data points

4.3. AI Chatbot (Context-Aware)

The Chatbot is now fully functional. Unlike a generic wrapper, our implementation includes:

- I. **Full-Context Integration:** Leveraging DeepSeek's extensive context window, we architected the system to retrieve and inject the patient's **entire health dataset** (profile, thresholds and historical logs) into the system prompt. This approach allows us to benchmark the model's reasoning capabilities on complete patient histories without the complexity of retrieval algorithms such as (RAG) at this stage.
- II. **Prompt Engineering:** We rigorously engineered the system prompt to define the "Florence" persona. This involved structuring raw JSON data into a readable format for the LLM and implementing strict behavioural instructions (e.g., "be empathetic," "never diagnose," "reference specific data points"), ensuring the AI remains safe and helpful even with large context inputs.

- III. **Architectural Convergence (Tool Calling):** Since the upcoming Automation Engine requires an agentic "Large Action Model" architecture, we plan to refactor the Chatbot to utilise **Tool Calling** (Function Calling) for data retrieval. We will conduct A/B testing to determine if dynamically fetching data via tools offers better performance and lower token usage compared to the current full-context injection method.
- IV. **Reliable State Management:** We implemented a persistent conversation state stored in Supabase (*patient_chat_history*), decoupling the chat logic from the UI lifecycle.
- Background Processing:** If a user exits the screen while the AI is generating a response, the backend service completes the processing and saves the message to the database independently.
 - Seamless Synchronisation:** When the user returns, the *ChatbotService* re-syncs the full conversation history. This eliminates common bugs that occur when returning to a chat, resulting in disjointed threads or missing context, and ensures a smooth, continuous user experience even across app restarts.

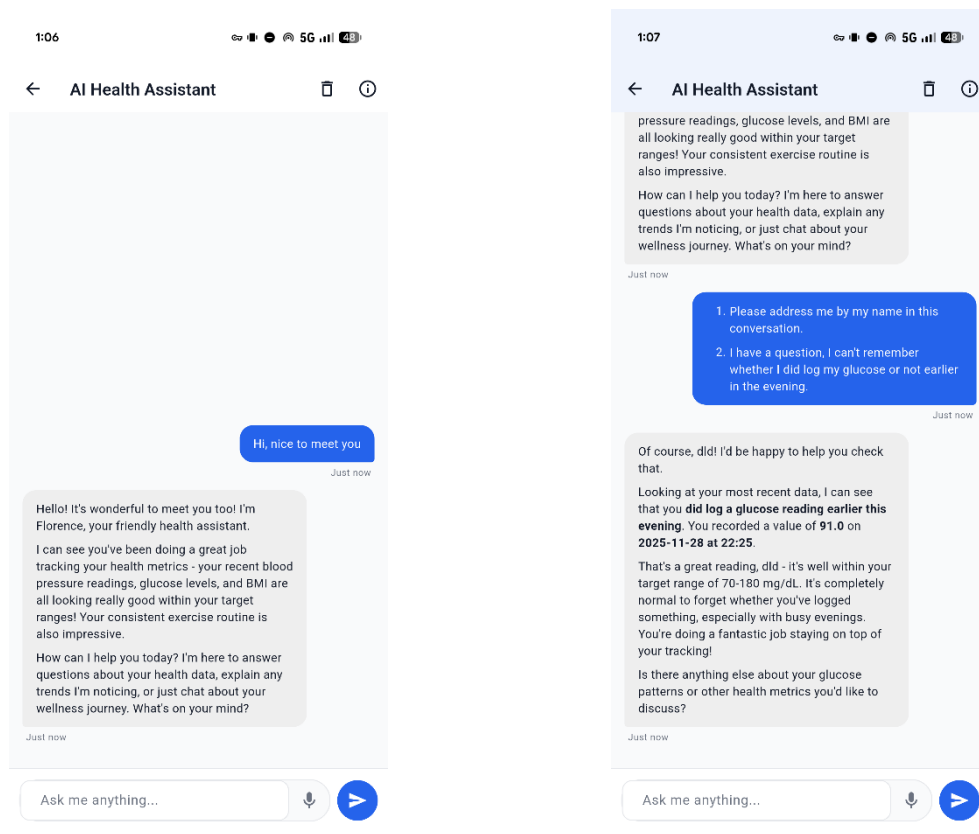


Figure 3: A chat conversation where the AI references the patient's specific glucose reading from the database

4.4. Authentication

We finalised the complete authentication architecture, migrating from development-only anonymous logins to a production-ready system.

- I. **Secure Registration:** Enforced email/password authentication using Supabase Auth with strict password policies, ensuring data security for both patients and clinicians.
- II. **Frictionless Verification (Deep Linking):** We implemented a seamless onboarding flow using deep links. When a user clicks the verification link in their email, the app launches immediately and automatically establishes an authenticated session. This eliminates the need for manual re-login after verification, significantly reducing user friction during the first run.

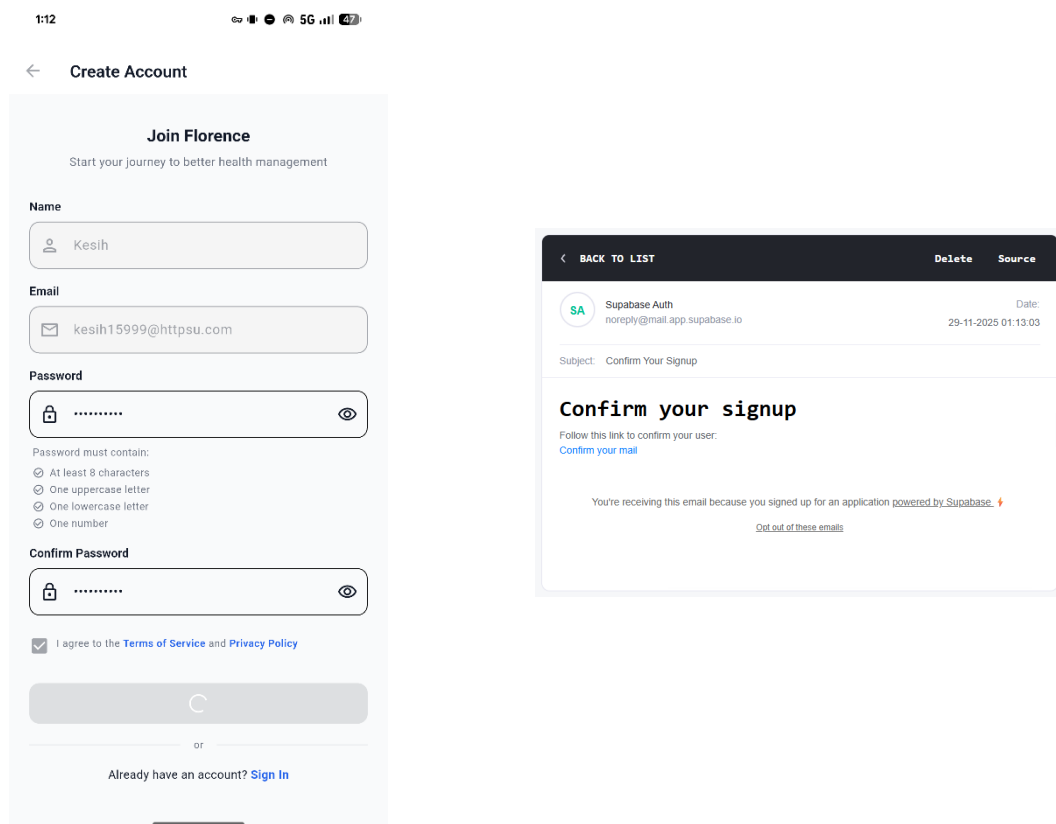


Figure 4: The Register Screen and the "Check your email" verification prompt

5. Sprint Review

The review process for this sprint deviated from the standard live demonstration format due to a strategic pivot directed by the project supervisor during our mid-sprint consultation.

5.1. Consultation and Strategic Realignment

In our consultation mid-sprint, we presented the initial UI mockups and architectural plan. The supervisor provided a clear directive: **prioritise the functional connectivity of the Patient Dashboard over the breadth of features.**

Based on this, the team dedicated the remainder of the sprint to intense development work to finalise the "Vertical Slice" of the Patient App.

- I. **Current Status:** The Patient Dashboard is now fully functional, connected to the backend and visually overhauled. Internal testing confirms that data logging, visualisation and the AI Chatbot are operating correctly.
- II. **Deployment Strategy:** To overcome the limitations of physical demonstrations, where testing is restricted to the short consultation time slot on a team member's device, we are transitioning to an Asynchronous User Acceptance Testing (UAT) model. **Immediately following this submission, we plan to establish a CI/CD pipeline (targeting Firebase App Distribution).** This will allow the supervisor to install the functional build on their own device and test it at their convenience, facilitating a much deeper review than a brief in-person session permits.

5.2. Client Feedback

The feedback driving this sprint's output was derived from the mid-sprint consultation:

- I. **Directive:** The client explicitly instructed the team to "finish the basic functionality for the patient dashboard first" before expanding to the Clinician side or advanced automation.
- II. **Scope Adjustment:** It was mutually agreed to defer the LLM Automation Engine to the next semester. The client emphasised that a solid, working data

platform is more valuable for the final presentation than a buggy experimental feature.

5.3. Critical Analysis

This sprint represents a massive leap in technical maturity, moving the project from "Design" to "Engineering."

- I. **Progress Against Plans:** We successfully achieved the sprint's primary goal: delivering a fully connected Patient App. While the Clinician Dashboard backend is ready, we deliberately paused its UX development to ensure the Patient side was polished and bug-free, strictly adhering to the client's quality-first directive.
- II. **Challenges and Solutions:**
 - a. **Complex State Synchronisation:** Managing state consistency across the Chatbot, Data Logging and Visualisation widgets proved to be the most significant technical hurdle. Initially, data logged in one screen would not immediately reflect in others due to reliance on local widget state. We resolved this by refactoring the entire app to use **Riverpod** for global state management, allowing us to invalidate specific data providers upon successful API writes to trigger instant, synchronised UI updates across the app.
 - b. **Clinical Domain Expertise for Visualisation:** We underestimated the domain knowledge required to implement medically useful visualisations. We learned that generic charting libraries are insufficient without customisation, for example, Blood Pressure requires visualising the correlation between Systolic and Diastolic pressure, whereas Glucose data requires highlighting "Time in Range." Bridging the gap between raw data rendering and clinical interpretability required significant research and custom implementation of the *fl_chart* library.
- III. **Outcome:** The system is stable. The separation of concerns (Frontend, Data Service, Chatbot Service) allows for independent scaling and easier debugging.

6. Retrospect (Critical Review of The Process)

6.1. Strengths

- I. **Microservice Architecture:** Separating the Chatbot logic into its own Python service (FastAPI) proved excellent. It allowed the AI engineers to tweak prompts and context logic without breaking the main mobile app.
- II. **Monorepo Strategy:** Continuing the strategy from Sprint 2, the monorepo allowed for easy sharing of data models and rapid iteration between frontend and backend teams.

6.2. Challenges and Bottlenecks

- I. **Clinician Workflow Optimisation:** While the Clinician Dashboard successfully integrates with the backend, the User Experience (UX) lacks clinical efficiency. The current "Patient Review" workflow introduces unnecessary friction and navigation steps. Significant iteration is required to streamline the interface, ensuring clinicians can assess patient risk profiles rapidly and intuitively.
- II. **Profile Data Complexity:** The patient profile schema proved more complex than anticipated, particularly due to the requirement for granular, personalised health thresholds for every user. Integrating this extensive data model with the frontend forms consumed significant development time. To maintain sprint velocity and ensure stability, we prioritised the display logic over editing capabilities, leaving secondary fields as read-only for this release.

6.3. Ethical and Security Review

- I. **Data Isolation:** We verified that Row Level Security (RLS) policies are active. A patient authenticated with Token A cannot query the data of Patient B.
- II. **No PII in LLM:** We ensured that while health metrics are sent to the LLM, no Personally Identifiable Information (PII) like real names or addresses are included in the prompt context to preserve privacy.

7. Lessons Learned (Critical Review of Sprint Two Experience and Future Plan)

7.1. Lessons Learned

- I. **Don't over-promise on AI:** We learned that "getting the data ready" for AI takes 80% of the time. The actual LLM integration is fast, but building the pipeline to feed it clean data is a slow process.
- II. **State Management is Key:** In a data-heavy health app, centralised state management (Riverpod) is not optional, it is essential to prevent UI bugs.
- III. **Authentication is Complex:** Deep linking and session management on mobile devices require significantly more testing time than web-based auth.

7.2. Recommendations for Future

- I. **Freeze Core Features:** No new "core" features should be added. The focus must now shift entirely to polishing and documentation.
- II. **Clinician UX:** The next immediate task is to smooth out the clinician's user journey to match the quality of the patient dashboard.

7.3. Plan for Next Semester

With the foundation complete, the roadmap for the next semester is clear:

- I. **Implement the LLM Automation Engine:** Now that we have reliable data ingress, we can build the background workers to trigger alerts.
- II. **Advanced Analytics:** Move from simple line charts to predictive trend analysis.
- III. **Final Polish:** Comprehensive error handling and UI animations.

This sprint successfully redeemed the project's trajectory by delivering a fully integrated, demonstrably working product.